M2-BBS UNIVERSITÉ PAUL SABATIER

Rapport - GDNS - Application post-génomique

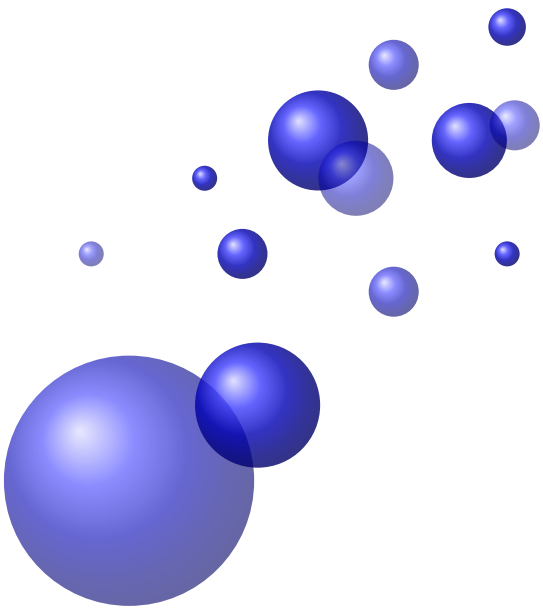
Caractérisation d'un ensemble de gènes via la création d'une application Python et d'une base de données Neo4j - set.M2.6.txt -

version corrigée

https://github.com/boumbo85/ecoli-neo4j-genomics

Auteure :
Julie CAMPOS

Sous la supervision de :
Dr. Roland BARRIOT



Novembre 2023

Table des matières

1 INTRODUCTION **2**
- 1.1 CONTEXTE . . . 1
- 1.2 OBJECTIF . . . 1
- 1.3 MÉTHODE . . . 2

2 CREATION DE LA BASE DE DONNÉES NEO4J **3**
- 2.1 PRÉSENTATION DES DONNÉES . . . 4
- 2.2 MODÉLISATION DE LA BASE DE DONNÉES . . . 5
- 2.3 PRÉREQUIS . . . 5
- 2.4 PRÉPARATION DES DONNÉES . . . 7
- 2.5 INTÉGRATION DES DONNÉES DANS LA BASE . . . 8
  - 2.5.1 MÉTHODE . . . 8
  - 2.5.2 DESCRIPTION . . . 9

3 APPROCHE PAR ENSEMBLE : ENRICHISSEMENT **11**
- 3.1 CRÉATION DE L'APPLICATION PYTHON . . . 12
  - 3.1.1 Description . . . 12
    - 3.1.1.1 Principe étape des tests statistiques : . . . 12
    - 3.1.1.2 Présentation générale des tests statistiques utilisés : . . . 12
  - 3.1.2 Utilisation . . . 14
- 3.2 RÉSULTATS APPLICATION PYTHON . . . 15
  - 3.2.1 Choix de la mesure de similarité . . . 15
  - 3.2.2 Bilan : choix des mesures . . . 16
  - 3.2.3 Synthèse des résultats (set 6) . . . 16
  - 3.2.4 Observations . . . 17
- 3.3 RÉSULTATS REVIGO . . . 18

4 INTERPRETATION **22**
- 4.1 [v2] VALIDATION ET COHÉSION DE COEXPRESSION . . . 23

5 CONCLUSION **25**
- 5.1 BIOLOGIQUE . . . 26
- 5.2 RETOUR CRITIQUES . . . 26
- 5.3 AMÉLIORATION . . . 26

5.3.1 Application 26
5.3.2 Tests statistiques 27
5.4 BILAN PERSONNEL 27

6 ANNEXE - SCRIPT R1 - TRANSFORMATION DONNÉES 29

7 ANNEXE - SCRIPT R2 - INTEGRATION DES FICHIERS CSV DANS LA BASE DE DONNÉES NEO4J 40

8 ANNEXE - SCRIPT PYTHON - APPLICATION ENRICHISSEMENT 58

9 ANNEXE - ENSEMBLE QUERY 6 - set.6.M2.txt 64

CHAPITRE 1

INTRODUCTION

Préambule : Nous choisissons d'écrire le rapport en français et les codes (R, Phyton) en anglais.

1.1 CONTEXTE

Les recherches en génomique ont permis de réaliser des progrès considérables dans l'élaboration de cartes physiques et génétiques et de déterminer la séquence de nombreux génomes. Elles ont ouvert la voie et grandement contribué à l'acquisition de nouvelles connaissances en physiologie par exemple. Parallèlement, le développement de nouvelles technologies d'analyses moléculaires à grande échelle (métagénomique, transcriptomique et protéomique) et l'essor de la bio-informatique rendent maintenant possible l'exploitation de ce flux de données et permettent par une étude globale de l'expression des gènes d'avoir une vision moins réductionniste du vivant. Ces progrès sont à l'origine d'une nouvelle discipline de la génomique appelée la "post-génomique" (ou génomique fonctionnelle). Elle vise à comprendre la fonction et la régulation de tous les gènes et protéines qui conditionnent le fonctionnement de l'organisme.
De nombreuses expériences de laboratoire requièrent une étape de séquençage d'échantillons (séquences nucléotidiques) et permettent ainsi de mettre en évidence une liste de gènes d'intérêts. Ces séquences sont obtenues par des expériences telles que : RNAseq, sRNAseq, CHIP-sep ou ATACseq. L'étape d'après consiste à interpréter et caractériser cette liste de gènes. C'est ici que la "post-génomique" entre en jeu. La "post-génomique" fait référence à la recherche et l'analyse en génomique survenant après le séquençage d'échantillons d'ADN ou d'ARN. Elle englobe divers domaines de recherches, notamment :

- **analyse fonctionnelle de gènes**
- **protéomique**
- **structure 3D des protéines**
- **épigénomique**
- **analyse des réseaux biologiques**
- **transcriptomique**
- **expression génique**
- **métagénomique**
- **évolution moléculaire**

C'est donc un enjeu majeur que de savoir caractériser une liste de gènes afin d'interpréter les résultats.

1.2 OBJECTIF

Cette étude vise donc à caractériser un ensemble de gènes, à partir d'une liste fournie d'identifiants (*bnumber*) de gènes (set.M2.6.txt). Nous répondrons entre autre à la question suivante : A quels processus biologiques, fonctions moléculaires ou localisations sub-cellulaires peut-on les relier ?
A travers cet exemple, l'objectif intrinsèque est d'implémenter une base de données non structurées (graphe, Neo4j) incluant toutes les informations relatives à un projet (par exemple les références "PubMed", nom des gènes, fonctions biologiques, voies métaboliques), et d'explorer ses nombreuses possibilités.

1.3 MÉTHODE

Nous présenterons ici une méthode pour étudier cette liste de gènes, s'appuyant sur la construction d'une base de données sous forme de graphe (**"Neo4j"**), et la création d'une application python permettant d'automatiser cette caractérisation. L'application prendra en entrée une liste de gènes d'intérêt (appelée *"query"*), ainsi qu'une liste de gènes de référence (appelée *"target"*). L'application retournera en sortie des statistiques descriptives, une liste de termes en fonction de l'option choisie (mots clef de fonctions biologiques, identifiants "GOTerms", références "Pubmed", voies métaboliques). Il s'agit d'une approche par ensembles d'enrichissement. Le but de cette méthode est de déterminer si les gènes interrogés (requête) présentent un enrichissement significatif dans certaines fonctions biologiques par rapport à ce qui serait attendu de manière aléatoire [1].

1. Nous suivrons la méthode proposée en travaux pratiqueshttps://silico.biotoul.fr/enseignement/m2bbs/gdns-ap/GDNS-APG.Mise.en.oeuvre.html#approche-par-ensembles-enrichissement

CHAPITRE 2

CREATION DE LA BASE DE DONNÉES NEO4J

Les script R pour l'intégration de la base de néo4j ont été élaboré conjointement avec Claire DELAMARE-DEBOUTTEVILLE.

2.1 PRÉSENTATION DES DONNÉES

L'échantillon d'étude est issu de la bactérie ***Escherichia.coli***(*E.coli*), souche **K-12 MG1655** (Ecocyc). La souche K-12 MG1655 est une souche très répandue en laboratoire [1]. *E.coli* est une entérobactérie, de type gram négatif. Elle est présente dans la flore intestinale des organismes homéothermes [2]. E.coli participe à la suppression de la croissance des bactéries nocives et synthétise des vitamines dans le tractus intestinal. La plupart des souches d'*E.coli* sont sont commensales, mais une minorité de souches peut provoquer des maladies chez l'homme.

E.*coli* possède un génome à ADN double brin circulaire de 4,6 millions de paires de bases, 4401 gènes (Uniprot) [3].

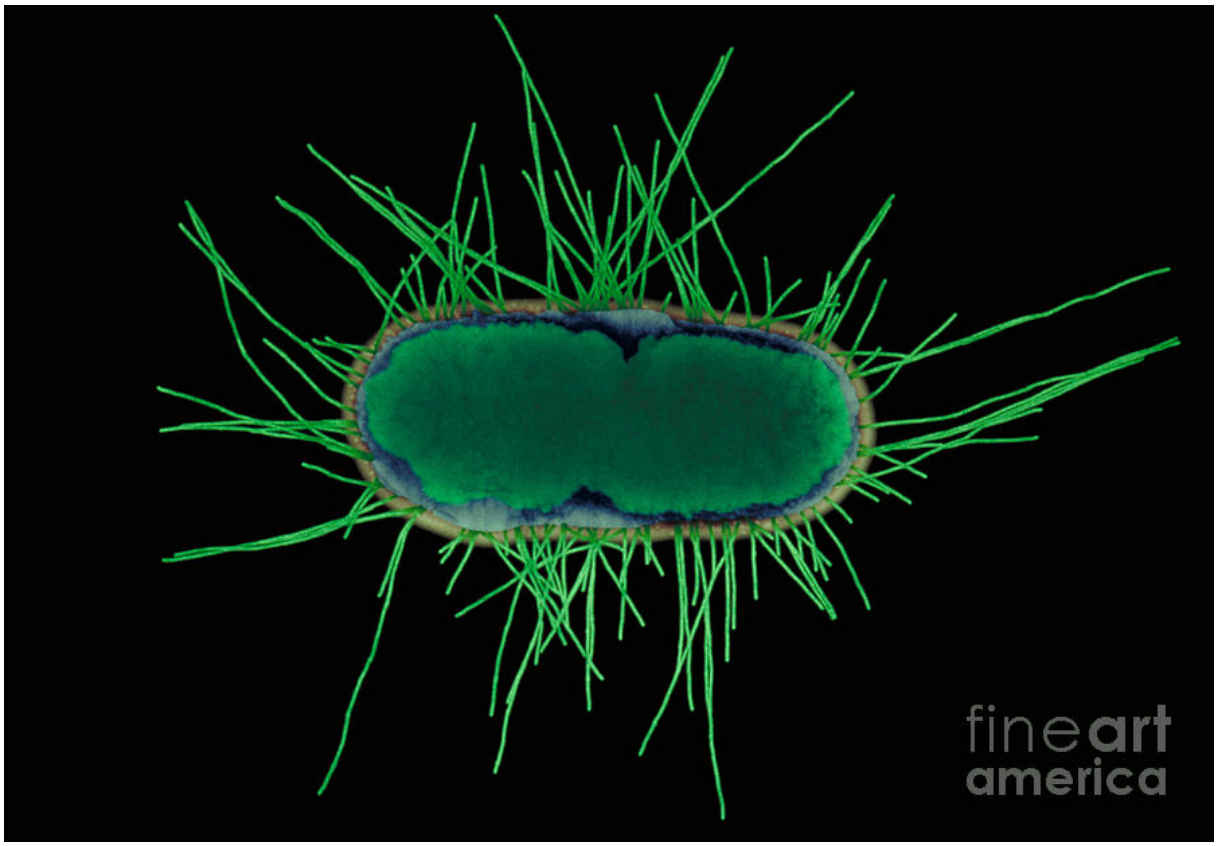


FIGURE 2.1 – Micrographie électronique à transmission (TEM) en couleur d'*Escherichia Coli* K12 - 2014 - by Kwangshin Kim

Nous travaillerons sur deux listes de gènes, une de test (set1)dont les résultats sont connus, pour mettre en place l'application , et la liste d'étude (set.M2.6, cf. 9) à analyser.

1. urlhttps ://www.ncbi.nlm.nih.gov/Taxonomy/Browser/wwwtax.cgi ?id=511145, https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9997739/
2. urlhttps ://fr.wikipedia.org/wiki/Escherichia_coli
3. urlhttps ://pubmed.ncbi.nlm.nih.gov/11574054/

Nous utiliserons comme génome de référence GCF_000005845.2, au format .gbff (fichier GCF_000005845.2.gbff.gz).

2.2 MODÉLISATION DE LA BASE DE DONNÉES

L'implémentation de la base de donnée **"Neo4j"** requière la création de nœuds et de relations (connexions entre les nœuds). Nous intégrerons dans la base de données entant que nœuds, les informations suivantes [4] :

- **Gene :** identifiants (*"bnumber"*,nom des gènes, sens du brin (*"strand"*) ainsi que leur position sur le(s) chromosome(s)
- **Alias :** les alias/synonymes des noms de gènes
- **Keyword :** identifiants *"Uniprot"* et mots clef de fonctions biologiques (*UniProt Proteome dataBase*)
- **InterPro :** identifiants *"Uniprot"* et nom des domaines protéiques (*UniProt Proteome dataBase*)
- **PubMed :** identifiants *"PubMed"* et nom des références bibliographiques (GenBank/RefSeq)
- **TU :** identifiants *"Ecocyc"*, nom, sens du brin, position de début et fin des unités de transcription. Pour rappel, une unité de transciption est une région du génome transcrit en ARN. Certains gènes peuvent coder pour plusieurs protéines (un isoforme = une unité de transcription)
- **Pathway :** identifiants *"Ecocyc"*, nom des voies métaboliques
- **GOTerm :** identifiants *"GOTerm"*, nom et description des termes de l'ontologie des gènes (*Gene Ontology + UniProt*)
- **StringDB :** Scores d'association entre paires de gènes/protéines basés sur la co-expression, le lien phylogénomique, et les interactions protéine-protéine.

Remarques :

1. Chaque nœud contient un identifiant unique (clef primaire). Donc chaque nœud correspond à une liste de termes uniques.
2. Selon les sources de données, les méthodes de référencement et d'identification peuvent ne pas être les mêmes. Dans cette étude, nous nous baserons sur les *bnumbers*(identifiant Uniprot). Le jeu sera d'associer à chaque identifiant du nœud, un *bnumbers*

Les relations entre les nœuds sont décrites d'après le schéma suivant : (*cf. fig.2.2*)

Les relations sont construites de tel sorte qu'il existe au minimum deux attributs : l'identifiant du nœud concerné et le *"bnumber"* associé. Chaque relation est donc constituée d'une liste unique d'associations **terme - *bnumber***.

2.3 PRÉREQUIS

Pour le projet, nous avons utilisé les outils informatiques et leur versions associées suivantes [5] :

4. url https ://silico.biotoul.fr/enseignement/m2bbs/gdns-ap/GDNS-APG.Mise.en.oeuvre.html#approche-par-ensembles-enrichissement partie 1.3

5. Pour plus de détails, cf. les fichiers "requirements" dans le github du projet : https ://github.com/boumbo85/ECOLI

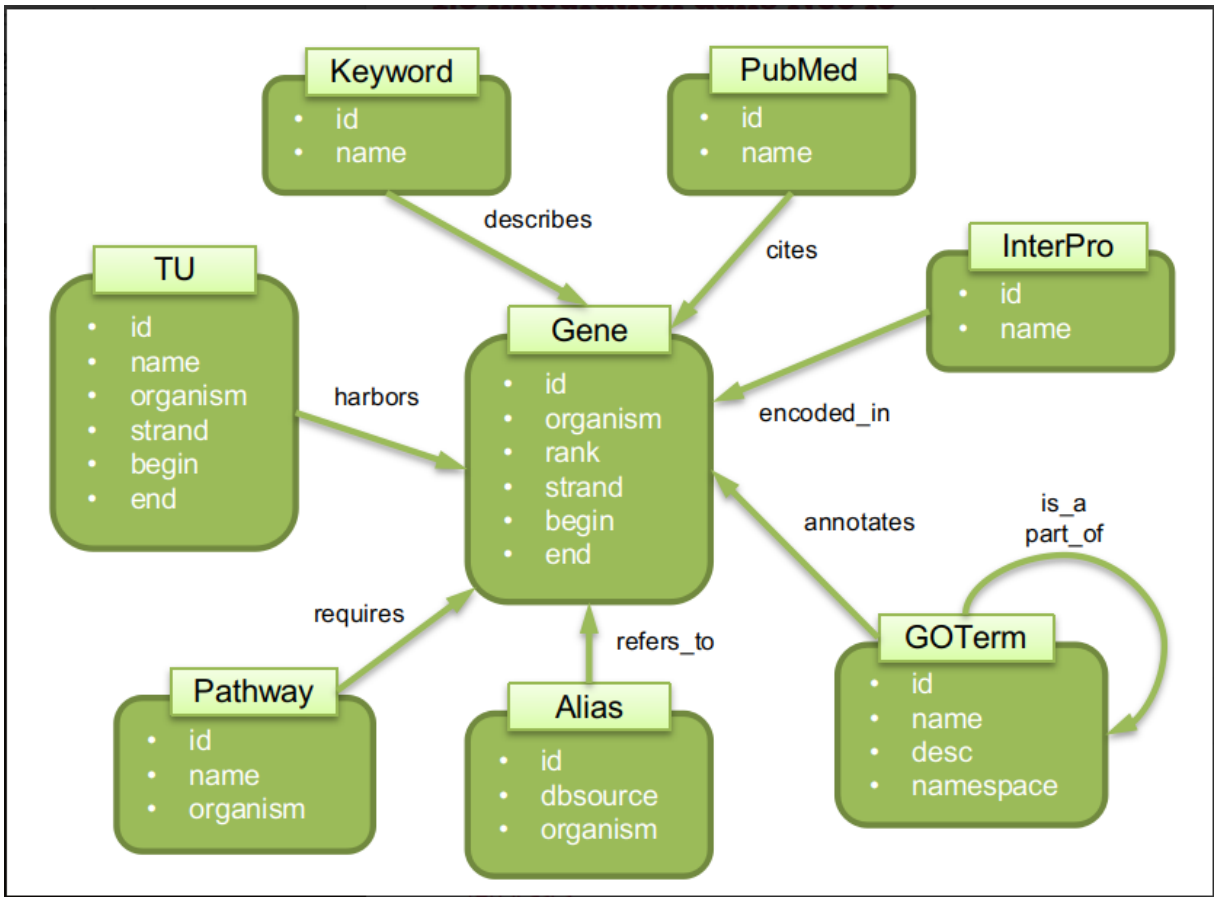


FIGURE 2.2 – Modélisation de la base de données Neo4j

— **Système d'exploitation (OS) :** Linux FEDORA → version release 37 (Thirty Seven) [6]

— **Logiciels :**

- **conda :** → version (23.3.1)
- **Neo4j Desktop :** → version (1.4.15)
- **Neo4j :** → version (4.4.9) (*Edition : community*)
- **R base :** → version (4.3.2)
- **Rstudio :** → version (2023.06.1 524) - "Mountain Hydrangea"

```
Release (547dcf861cac0253a8abb52c135e44e02ba407a1, 2023-07-08)
for Fedora release 37 (Thirty Seven)
Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
QtWebEngine/5.15.12 Chrome/87.0.4280.144 Safari/537.36
```

- **Python :** → version (3.12.0)
- **VScode :** → version (1.83.1)

— **Package Python (3.12.0) :**

- pandas → version (2.1.2)
- numpy → version (1.26.1)
- monotonic → version (1.6)
- py2neo → version (2021.2.4)
- neo4j → version (5.14.0)
- scipy → version (1.11.3)
- matplotlib → version (3.8.1)
- argparse (parso → version 0.8.3)

— **Package R (4.3.2) :**

- tidyverse (CRAN) → version (2.1.2)
- tibble (CRAN) → version (3.2.1)
- dplyr (CRAN) → version (1.1.3)
- jsonlite (CRAN) → version (1.8.7)
- RMySQL (CRAN) → version (6.1.11)
- neo4r (CRAN)→ version (0.1.1)
- reticulate (CRAN) → version (1.34.0)
- STRINGdb (BioConductor) → version (2.12.0)
- igraph → version (1.5.1)

2.4 PRÉPARATION DES DONNÉES

(cf. ANNEXE 6 et https://silico.biotoul.fr/enseignement/m2bbs/gdns-ap/GDNS-APG.Mise.en.oeuvre.html#approche-par-ensembles-enrichissement
Le but ici est de créer un fichier au format .csv par nœud et par relation. Chaque nœud est une liste unique de termes (selon les catégories d'informations retenues (Goterm, Keywords, Uniprot..). Chaque terme est associé à un identifiant unique (clef primaire) et une sélection

6. lsb_release -a

d'informations relatives à cet identifiant. Chaque relation est une liste d'association unique de ***"bnumber"*** et termes. Voici comment nous construisons les fichiers .csv.

Nous récupérons le protéome d'*Escherichia.coli* K12 MG1655, au format .tsv sur le site d'Uniprot [7]. Nous choisissons comme référence de notation les *"bnumber"*. A partir de ce fichier, nous pouvons extraire les listes de :

- ***"bnumber"* :** à partir de la colonne "Gene Names (ordered locus)" (liste des identifiants selon plusieurs système de notations) :
  4401 lignes (uniques)
- **Goterm :** à partir de la colonne "Gene Ontology IDs" (liste des Goterms) :
  22430 lignes (uniques)
- **Keywords** à partir de la colonne "Keywords" (liste des mots clef des fonctions biologiques) :
  27506 lignes (uniques)
- **InterPro** à partir de la colonne "InterPro" (liste des identifiants InterPro correspondant à un domaine/motif structural) :
  15636 lignes (uniques)

associées à chaque nom de gène. Nous obtenons une liste d'association unique de *"bnumber"* et de terme (**Keywords**, **InterPro** et **Goterm**).

Les unités de transcription (**TU**) [8] et les voies métaboliques (**Pathway**) [9] sont disponibles sur le site d'Ecocyc (https://www.ecocyc.org/). Cette fois-ci, nous devons faire correspondre l'identifiant "Ecocyc" avec l'identifiant "Uniprot", les *"bunmber"*. Nous trouvons cette correspondance dans la table "gènes" [10].

Les identifiants **PubMed** sont extrait via une application python fournie.

2.5 INTÉGRATION DES DONNÉES DANS LA BASE

(cf. ANNEXE7

2.5.1 MÉTHODE

Nous utilisons une image docker de Neo4j(neo4j :4.4.9-community) pour se connecter avec "podman" (linux Fedora), avec la commande "BASH" suivante :
podman pull neo4j :4.4.9-community

Cela permet d'utiliser la version de Neoj que l'on souhaite et évite d'installer Neo4j Desktop. Il s'agit maintenant de connecter le docker avec notre session locale, avec la commande "BASH" suivante :
podman run –rm -it –name neo4j
-p 7474 :7474
-p 7687 :7687
-v ./neo4j.data :/data
-v ./neo4j.import :/import
-e NEO4J_AUTH=neo4j/omybioinfo

7. https://www.uniprot.org/
8. https://www.ecocyc.org/group?id=biocyc17-55140-3842483291
9. https://www.ecocyc.org/group?id=biocyc17-55140-3842483872
10. https://www.ecocyc.org/group?id=biocyc13-55140-3842501533

–user=”$(id -u) :$(id -g)”
neo4j :4.4.9-communit
La base de donnée implémentée avec succès se présente donc ainsi : (*cf. fig.2.3*)

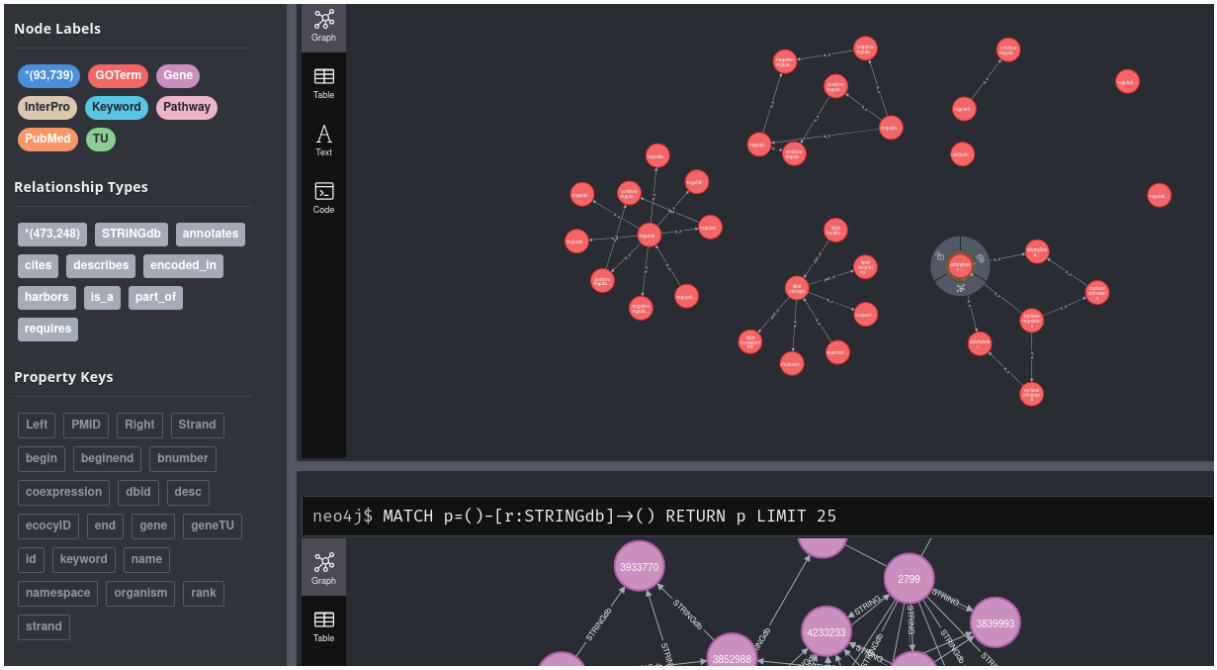


FIGURE 2.3 – Aperçu de la base de donnée Neo4j

2.5.2 DESCRIPTION

La base de données contient , incluant :

— **7 nœuds** tels que :
- 4 315 ”**Genes**”
- 385 ”**Keywords**”
- 6 760 ”**InterPros**”
- 42 887 ”**GOTerms**”
- 3 634 ”**TUs** ”
- 438 ”**Pathways**”
- 35 320 ”**PubMeds**”

— **9 relations** telles que :
- 27 323 ”**describes**” (relation ”**Keyword**” → ”**Gene**”
- 101 841 ”**cites** ” (relation ”**PubMed**” → ”**Gene**”
- 15 515 ”**encoded_in**” (relation ”**InterPro**” → ”**Gene**”
- 68 521 ”**is_a**” (relation ”**GOTerm**” → ”**GOTerm**”
- 6 785 ”**part_of**” (relation ”**GOTerm**” → ”**GoTerm**”
- 22 373 ”**annotates**” (relation ”**GOTerm**” → ”**Gene**”

- 6 170 "**harbors**" (relation "**TU**" → "**Gene**"
- 2 747 "**requires**" (relation "**Pathway**" → "**Gene**"
- 378 893 "**STRINGdb**" (gènes co-exprimés)

CHAPITRE 3

APPROCHE PAR ENSEMBLE : ENRICHISSEMENT

3.1 CRÉATION DE L'APPLICATION PYTHON

3.1.1 Description

Le script python de l'application d'enrichissement a été élaboré conjointement avec Claire DELAMARE-DEBOUTTEVILLE. *(cf. ANNEXE 8)* et le fichier 'app.py'.
Le script d'enrichissement prend en entrée un ensemble de gènes requêtes (ici le "set 6"). Le "set 6" est une liste d'identifiant de gènes ***"bnumber"***. Le format de la liste du "set 6" a besoin d'être transformée afin d'être conforme au format d'entrée de notre application. En effet, la liste "set 6" contient un identifiant par ligne. Or il faudrait que les identifiants soient séparés d'un espace standard et non d'un saut de ligne (modification via la commande "BASH" : tr -s '/n' ' ' < query6 > query6.txt). Le script permet de :

- se connecter à la base de données Neo4j (BD) nouvellement créée
- d'interroger celle-ci via des requêtes "CYPHER" : récupérer les termes/mot clefs associés à la liste de ***"bnumber"*** de gène de l'ensemble requête.
- réaliser une analyse univariée (statistiques descriptives) sur la liste de gènes inconnus et les mots clefs collectés
- comparer l'ensemble de termes requêtes (*"query"*) avec l'ensemble de terme de référence (*"target"*) pour identifier les éléments en commun ('coverage')
- réaliser des tests statistiques avec plusieurs méthodes (binomiale, hypergéométrique, test du χ^2) sur chacun des termes (p-valeur ajustée aux multi tests ou non) afin de déterminer si cette différence est statistiquement due au hasard ou non (mesure de dissimilarité)

3.1.1.1 Principe étape des tests statistiques :

Nous chercherons à savoir si les différences observées entre l'ensemble requête et de référence sont statistiquement significatifs. Pour cela, nous suivrons les étapes classiques dans l'exécution de tests statistiques :

1. Définir les hypothèses de départ H0 (nulle) et H1 (alternative)
2. Choisir un test statistique pour contrôler H0
3. Définir la distribution de la statistique sous l'hypothèse H0 (est réalisée)
4. Fixer valeur seuil alpha (5%)
5. Calculer les probabilités (selon la distribution H0)à partir des données de l'échantillon
6. Conclure : si probabilité (p-valeur) supérieure au seuil alpha, nous acceptons HO et inversement, probabilité (p-valeur) inférieur au seuil alpha, nous rejetons H0 avec un risque 'erreur de 5%

3.1.1.2 Présentation générale des tests statistiques utilisés :

Nous cherchons à comparer deux groupes. Il s'agit de mesurer la différence entre deux ensembles de termes : ensemble requête (Q, *"query"*) vs ensemble de référence (T, *"target"*). Les deux ensembles sont inclus dans l'ensemble des gènes de l'organisme G (genome). Ils ont respectivement des cardinalités q, t, g. L'intersection des ensembles Q et T a une cardinalité c [1]. La cardinalité fait référence au nombre d'éléments contenu dans un ensemble donné.
Définition mathématique d'une cardinalité : "deux ensembles ont la même cardinalité (ou le

1. https://silico.biotoul.fr/p/M2BBS_GDNS-APG_-_Projets

même cardinal) s'ils sont en bijection" [2].

Il s'agit de variables quantitatives. Nous réaliserons des tests d'adéquation/ajustement : est ce que la distribution observée suit une loi/distribution de probabilité due au hasard, ou bien est elle singulière ? Le test compare la distribution observée avec la distribution théorique (par exemple, une distribution binomiale) et détermine si les différences sont significatives.

Les hypothèses considérées sont :

— **H0 :** Il n'y a pas de différence entre la distribution des termes de la requête par rapport à ce qui serait attendu au hasard (*"target"*). Autrement dit, la différence observée n'est pas significative et est due aux fluctuations d'échantillonnages. L'ensemble requête est comparé aux loi/distribution théorique de probabilité discrète suivantes :
  - loi binomiale
  - loi hypergéométrique
  - loi du χ^2 (pas dans un premier temps. nous ferons d'abord un test de χ^2 d'indépendance)

— **H1 :** rejet de H0, la différence entre les deux ensembles est significative

Nous considérerons les 2 ensembles Q et T comme 2 échantillons aléatoires, avec remise (loi binomiale) ou sans remise (loi hypergéométrique), et de calculer la probabilité d'avoir au moins autant d'éléments C en commun (par hasard) [3] . Il est utilisé pour calculer la probabilité d'obtenir k succès dans un échantillon de taille (n, 'query_size'), tiré d'une population totale de taille (N, _gene_total), contenant gènes associés (len(elements)). Il mesure la probabilité que k gènes en commun soient tirés de manière aléatoire.

Voici les paramètres requis :

— **k :** nombre d'éléments commun entre Q et T → nombre de succès observés

— **q :** nombre d'éléments de la requête Q ('query_size') → nombre d'essais (n)

— **t :** nombre d'éléments de la référence T ('len(elements)')

— **n_gene_total :** population → nombre total de gènes (N)

— **p :** la probabilité d'obtenir un gène associé aléatoirement selon la loi/distribution choisie → probabilité de succès

Test du χ^2 d'indépendance : Un test du χ^2 d'indépendance est utilisé pour évaluer si deux variables qualitatives (deux ensembles Q et T) sont indépendantes l'une de l'autre. Il examine si la fréquence observée des associations entre les deux variables diffère significativement de ce à quoi on pourrait s'attendre si elles étaient indépendantes (valeurs théoriques). Soit un tableau de contingence entre deux variables qualitatives Q et T sous l'hypothèse d'indépendance, lequel est obtenu en effectuant le produit des fréquences marginales, calculées telles que :

$$\text{Fréquence théorique/attendue} = \frac{\sum_i f(i) \cdot \sum_j f(j)}{f(t)} \tag{3.1}$$

avec
f(i) = fréquences marginales de la ligne i = Somme des fréquences observées dans la ligne i
f(j) = fréquences marginales de la colonne j = Somme des fréquences observées de la colonne j
f(t) = Somme totale de toutes les fréquences observées dans le tableau de contingence

2. https://www.lalanguefrancaise.com/dictionnaire/definition/cardinalite
3. https://silico.biotoul.fr/p/M2BBS_GDNS-APG_-_Projets

— **H0 :** Les deux variables sont indépendantes (hypothèse nulle).

— **H1 :** Il existe une relation significative entre les deux variables (hypothèse alternative)

Le test calcule ensuite une statistique du χ^2 qui mesure la différence entre les fréquences observées et les fréquences théoriques. Plus la statistique du χ^2 est élevée, plus il est probable que les variables ne sont pas indépendantes (H1). Si les deux distributions sont identiques, les variables Q et T sont indépendantes.

Option de mesure 'coverage' : Nous proposons une approche naïve qui consiste à considérer quelles fractions des 2 ensembles se correspondent. Pour la cohérence avec les autres mesures (et faciliter le tri de résultats de manière croissante), la valeur sera de 1 quand aucun élément n'est commun aux 2 ensembles, et 0 pour 2 ensembles identiques. Il s'agit donc de multiplier c/q par c/t avec c le nombre d'éléments communs aux ensembles Q et T respectivement de cardinalités q et t, et de soustraire cette valeur à 1 [4] :

$$proba = 1 - (k/query_size) * (k/t) \tag{3.2}$$

Cette mesure est de 1 quand aucun élément n'est commun aux deux ensembles (k=0), et elle diminue à mesure que la proportion d'éléments communs (k) augmente par rapport aux tailles des ensembles Q et T (q et t).

3.1.2 Utilisation

Nous utiliserons l'application python avec argparse. Cette fonctionnalité permet de lancer le script "Python" avec "BASH" et d'ajouter des arguments lors de l'appel de la fonction. Nous devons renseigner obligatoirement les arguments suivants :

— **'-q' ou '–query' :** le chemin du fichier contenant la liste de ***"bnumber"*** (query6.txt)

— **'-t' ou '–type' :** choix du type de mot clefs à collecter dans la BD parmi les nœuds suivant : 'Keyword', 'Pathway','TU','GOTerm','InterPro','PubMed'

Nous avons définit des valeurs par défaut pour certain arguments. Nous pouvons choisir de renseigner :

— **'-a', '–alpha' :** le seuil alpha de la p-valeur (par défaut : seuil 0.05 ou 5%)

— **'-c', '–adjust' :** application d'une correction des p-valeurs lors de test statistiques multiples (FDR). (par défaut : False). Si nous voulons utiliser la correction FDR, nous devons juste indiquer -c lors de l'appel de l'application python

— **'-m', '–measure' :** permet de choisir un type d'indice de dissimilarité parmi : 'binomial' (default), 'hypergeometric', 'chi2' or 'coverage' (choix du test statistique effectué)

— **'-l', '–limit' :** avec une valeur numérique, donne une limite d'affichage des résultats (par défaut : 0)

— **'-v', '–verbose :** affichage supplémentaire des résultats à chaque étape du script.

Exemple d'utilisation :
1. Définir le chemin du fichier requête et l'enregistrer dans une variable
path = '../query/query6.txt'
2. Lancer la commande "BASH" suivante :
python app.py
-q path
-t GOTerm

4. https://silico.biotoul.fr/p/M2BBS_GDNS-APG_-_Projets

-c -v -l 10
-m chi2 > Results/res.keyword.binomial.adj.txt

Ainsi, à partir d'une liste de gènes inconnus (liste de *"bnumber"* dans le fichier query6.txt) nous allons récupérer une liste de mots-clés (GOTerm) statistiquement enrichis (au seuil de 0.5%) associés à ces gènes. La mesure de dissimilarité s'effectuera avec le test 'binomial'. Avec cet exemple, seul les 10 premiers résultats seront affichés.
Avec la liste de GOterm obtenus, nous pourrons analysez les résultats du script (GOTerm et p-valeurs) avec REVIGO : http://revigo.irb.hr/.

3.2 RÉSULTATS APPLICATION PYTHON

3.2.1 Choix de la mesure de similarité

Voici quelques règles mathématiques révélant les condition d'utilisation des tests statistiques de l'application.

Test du χ^2 d'indépendance : Il est très rare en pratique, même dans le cas de variables réellement indépendantes, d'observer une égalité des distributions théoriques et empiriques et cela pour deux raisons :
1/ du fait que nous observons un échantillon et non pas la population entière 2/ à cause des erreurs de mesures.
Il existe d'autres conditions d'application pour le test du χ^2 d'indépendance : l'effectif total de la population doit être supérieur à 50, et les fréquences toutes supérieures à 5. Il existe des corrections (par exemple celle de Yates ou les tests exacts de Fisher et de Fisher-Freeman-Halton) [5]. Ce test ne tient pas compte d'un ordre éventuel sur les lignes et colonnes du tableau de contingence. Si nous pouvons ordonner les modalités de l'un des deux facteurs, nous préférerons utiliser un test de Kruskal-Wallis et si nous pouvons ordonner les modalités des deux facteurs on utilisera un test de Jonckheere-Terpstra.

Hypergeometrique :
Ce test est utilisé en général avec de faibles effectifs mais il est valide pour toutes les tailles d'échantillons. C'est un test qualifié d'exact car les probabilités peuvent être calculées exactement plutôt qu'en s'appuyant sur une approximation qui ne devient correcte qu'asymptotiquement (valeur à l'infini) comme pour le test du χ^2. Lorsque les effectifs observés sont relativement importants, il devient difficile d'évaluer de manière exacte la p-valeur du test.

Binomiale :
Lorsque N tend vers l'infini, la loi hypergéométrique converge vers une loi binomiale de paramètres n et p. D'ailleurs, intuitivement, pour N grand, tirer simultanément n boules revient à effectuer n fois une épreuve de Bernoulli dont la probabilité de succès serait p (p est la proportion de boules gagnantes dans l'ensemble des boules), car il est très peu probable de retomber sur la même boule, même si on la replace dans l'urne.

En pratique, on peut approcher la loi hypergéométrique de paramètres (n , p , N) par une loi binomiale de paramètres (n , p) dès que n / N $<$ 0,1, c'est-à-dire lorsque l'échantillon n est 10 fois plus petit que la population N. Or, dans notre cas, nous avons n = 113 (q, le nombre de *"bnumber"* dans le "set 6") et N = 4315 (le nombre total de gènes chez *E.coli* K12) $\rightarrow$ 0.026,

5. cours de statistique faculté de Rennes

ou quand n est inférieur à N/10. C'est le cas ici, cette approximation est donc légitime. Nous pouvons utiliser la loi binomiale pour notre étude.

3.2.2 Bilan : choix des mesures

Comme nous sommes dans le cas où n (113) est inférieur à N/10 (431), nous pouvons utiliser le test de conformité à la loi binomiale de préférence à la loi hypergéométrique. De plus, le test du χ^2 d'indépendance est trop approximatif (compare avec des fréquences théoriques jamais réalisable) et la mesure 'coverage' a une valeur descriptive, non statistique.
Pour cette étude il est préférable donc d'utiliser le test de conformité à la loi binomiale. La remise n'affecte pas les performances de l'application, étant donnée que n est petit.
Remarque : Nous obtenons les mêmes résultats avec les 3 tests (*cf. tableau 3.2.3 Synthèse des résultats*).

3.2.3 Synthèse des résultats (set 6)

(cf. tableau 3.2.3), https://github.com/boumbo85/ECOLI

Il y a 4315 gènes au total ('population size'). Et l'emsemble requête ("set 6", *"query"*) contient 113 gènes (*"bnumber"*).
Voici les 12 mots clefs des fonctions biologiques trouvés ('Keyword', 'binomial adjusted') :

- réponse au stress (*'Stress response'*)
- protéines de liaison à la GTP (*'GTP-binding'*)
- protéines chaperons (*'Chaperone'*)
- biosynthèse des lipopolysaccharides (*'Lipopolysaccharide biosynthesis'*)
- protéines de fimbrium (*'Fimbrium',cf. figure 2.1*). Le pilus (pili au pluriel) est un terme générique désignant deux types d'appendices de de la paroi de nombreuses bactéries à Gram négatif : les fimbriae et les pili sexuels. Ce sont des structures protéiques constituées de sous-unités peptidiques appelées pilines. Les fimbriae sont impliqués dans l'adhésion des bactéries sur des surfaces. Ils permettent par exemple l'attachement des bactéries sur les cellules eucaryotes. Plus courts et plus fins que des flagelles, ils ne peuvent pas être impliqués dans la mobilité.
- biosynthèse des Fimbrium (*'Fimbrium biogenesis'*)
- synthèse des exopolysaccharides (*'Exopolysaccharide synthesis'*). Un exopolysaccharide est un polymère de sucre secrété à l'extérieur de la cellule bactérienne.
- protéines de liaison aux nucléotides (*'Nucleotide-binding'*)
- magnésium (*'Magnesium'*)
- transférase (*'Transferase'*)
- Transduction sensorielle (*'Sensory transduction'*). La transduction sensorielle est l'étape de codage des informations du monde extérieur par des récepteurs sensoriels.

Voici les 8 mots clefs des fonctions biologiques trouvés ('GOTerm', 'binomial adjusted') avec l'application en ligne (plus visuel que les 232) :

- réponse à la chaleur (*'response to heat'*)
- activité diguanylate cyclase (*'diguanylate cyclase activity'*) impliqué dans la formation d'un biofilm bactériens
- activité phosphodiestérase spécifique du guanylate cyclique (*'cyclic-guanylate-specific phosphodiesterase activity'*) impliqué dans la formation d'un biofilm bactériens

Test	Keyword	InterPro	GoTerm	GoTerm(online) [7]	Pubmed	TU	Pathway
binomial	24	309	232	42	2233	145	11
binomial adjusted	12	213	232	8	2232	125	3
hypergeometric	24	309	232	42	2233	125	11
hypergeometric adjusted	12	309	233	9	2232	125	3
chi2	24	309	751	61	2233	145	11
chi2 adjusted	12	309	113	29	2206	142	4
coverage	24	309	145	-	2233	145	11

TABLE 3.1 – Synthèse des résultats obtenus - "set 6"- effectif

- biosynthèse des lipopolysaccharides (*'Lipopolysaccharide biosynthesis'*, LPS) constitue la membrane externe des bactéries à gram négatifs (OM) → rôle structural
- régulation négative de la motilité cellulaire dépendante du flagelle de type bactérien (*'negative regulation of bacterial-type flagellum-dependent cell motility'*).
- Activité ATPase (*'ATPase activity'*)
- l'adhésion biologique (*'biological adhesion'*)
- liaison des protéines non repliées (*'unfolded protein binding'*)
- processus de biosynthèse de l'acide colanique(*'colanic acid biosynthetic process'*). Dans leur environnement naturel, les microorganismes sont attachés à une surface, organisés en communautés structurées, et englobés dans une matrice d'exopolysaccharide, appelé biofil. L'acide colanique constitue l'enveloppe muqueuse du biofilm. La production est augmentée lors de l'attachement [6].
- repliement des protéines médié par les chaperons nécessitant un cofacteur (*'chaperone mediated protein folding requiring cofactor'*). Terme trouvé avec les options de recherche *'hypergeometric'*, *'adjusted'* sur le site 'REVIGO'.

3.2.4 Observations

(cf. tableau 3.2.3), https://github.com/boumbo85/ECOLI

— Les résultats obtenu sont sensiblement les mêmes avec la loi binomiale et hypergéométrique. Ce qui confirme la théorie mathématique précédente

— Les résultats obtenu avec l'application en ligne [8] sont plus restrictifs que ceux obtenu avec notre application. Je suppose que cela vient du fait que nous récupérons les **"GOTerm"** descendants, et pas seulement ceux directement associés. Les **"GOTerm"** associés sont plus génériques. Les termes trouvés entre les deux applications ont l'air cohérent dans leur fonction. Cette observation indique qu'il n'y a probablement pas d'erreur dans notre fonction.

— **[Correction v2]** Les résultats obtenus avec et sans ajustement des p-valeurs étaient initialement sensiblement les mêmes, ce qui ne devrait pas arriver (par exemple avec le set 1, options "Keyword" et "binomial", 41 sorties non ajustées contre 35 avec FDR, mais pour le set 6 et le set 4 de Claire, le même nombre de sorties). Nous avions supposé une erreur dans l'application : c'était le cas. La correction de Benjamini-Hochberg doit mettre à l'échelle la p-valeur par le **nombre de tests effectués** (nombre de termes testés, m), or

6. https://www.medecinesciences.org/en/articles/medsci/full_html/2003/01/medsci2003191p77/medsci2003191p77.html
8. https://silico.biotoul.fr/enrichment/

notre code la mettait à l'échelle par la **taille de l'ensemble requête** (le nombre de gènes). La formule correcte est $q_{(i)} = p_{(i)} \times m/i$ (où i est le rang de la p-valeur triée croissante), avec contrainte de monotonie. Le code corrigé utilise la bibliothèque `statsmodels` (implémentation de référence), avec un repli pur-Python équivalent, et des tests unitaires fixent les q-valeurs attendues.

3.3 RÉSULTATS REVIGO

(cf. revigo. irb. hr, Revigo version 1.8.1) L'ontologie des gènes classe les termes en 3 grands domaines (système OBO) : processus biologique, composant cellulaire, fonction moléculaire. Nous présenterons la liste des "GOterm" obtenus via l'application en ligne (silico.biotoul) avec les options χ^2 non ajustée. Nous trouvons 42 termes dont :

— **29** termes répertoriés dans les processus biologiques *(cf. figure 3.3)*

— **4** termes répertoriés dans les composants cellulaires *(cf. figure 3.3)*

— **23** termes répertoriés dans les fonctions moléculaires *(cf. figure 3.3)*

Voici les résultats de la similarité sémantique (option SimRel par défault, à partir de la base de données 'UNIPROT') : *(cf. figure 3.3, figure 3.3 et figure 3.3)*

Les 4 termes impliqués dans la structure cellulaire sont : *(cf. figure 3.3*

— membrane externe cellulaire

— pilus

— GroEL-ES complexe : protéines chaperonnes impliquée dans le repliement de nombreuses protéines [9]

— face externe de la membrane plasmique

Certains terme GO de la liste sont obsolètes, ou en doublons : *(cf. figure 3.3*

9. https://fr.wikipedia.org/wiki/GroEL

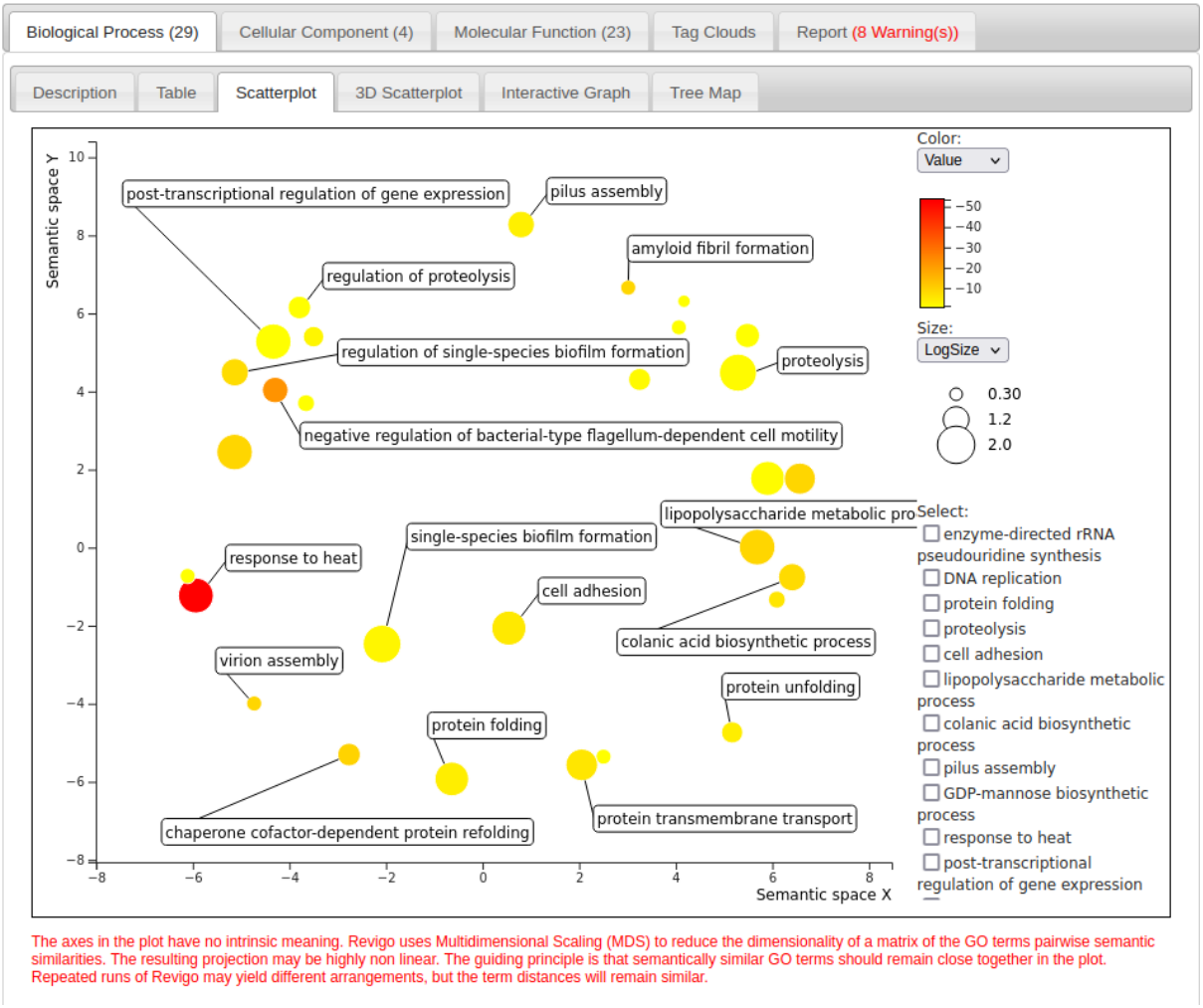


FIGURE 3.1 – Similarité sémantique des "GOTerm" - Processus biologiques - χ^2 non ajustée - set 6 - REVIGO

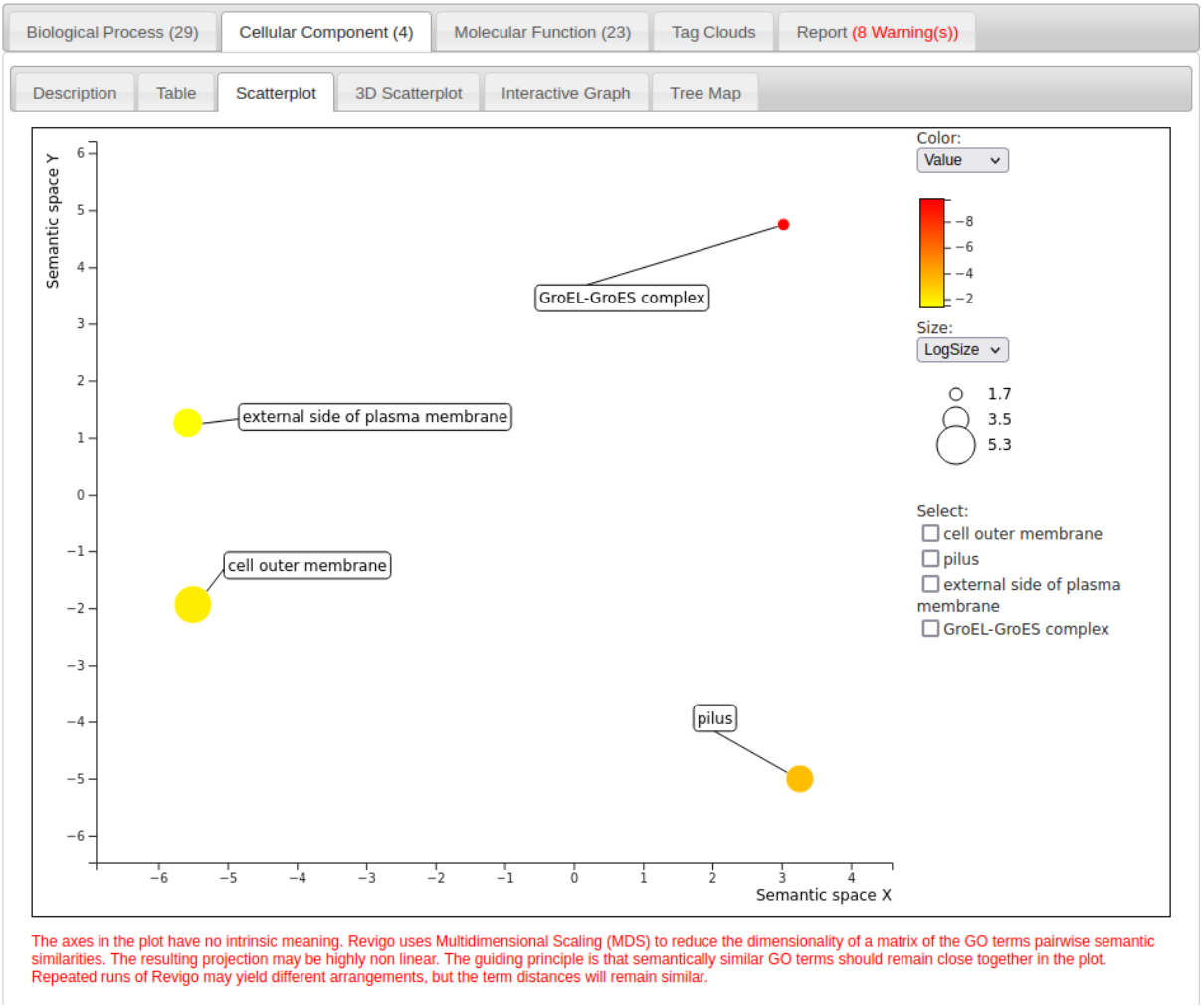


FIGURE 3.2 – Similarité sémantique des "GOTerm" - Composants cellulaires - χ^2 non ajustée - set 6 - REVIGO

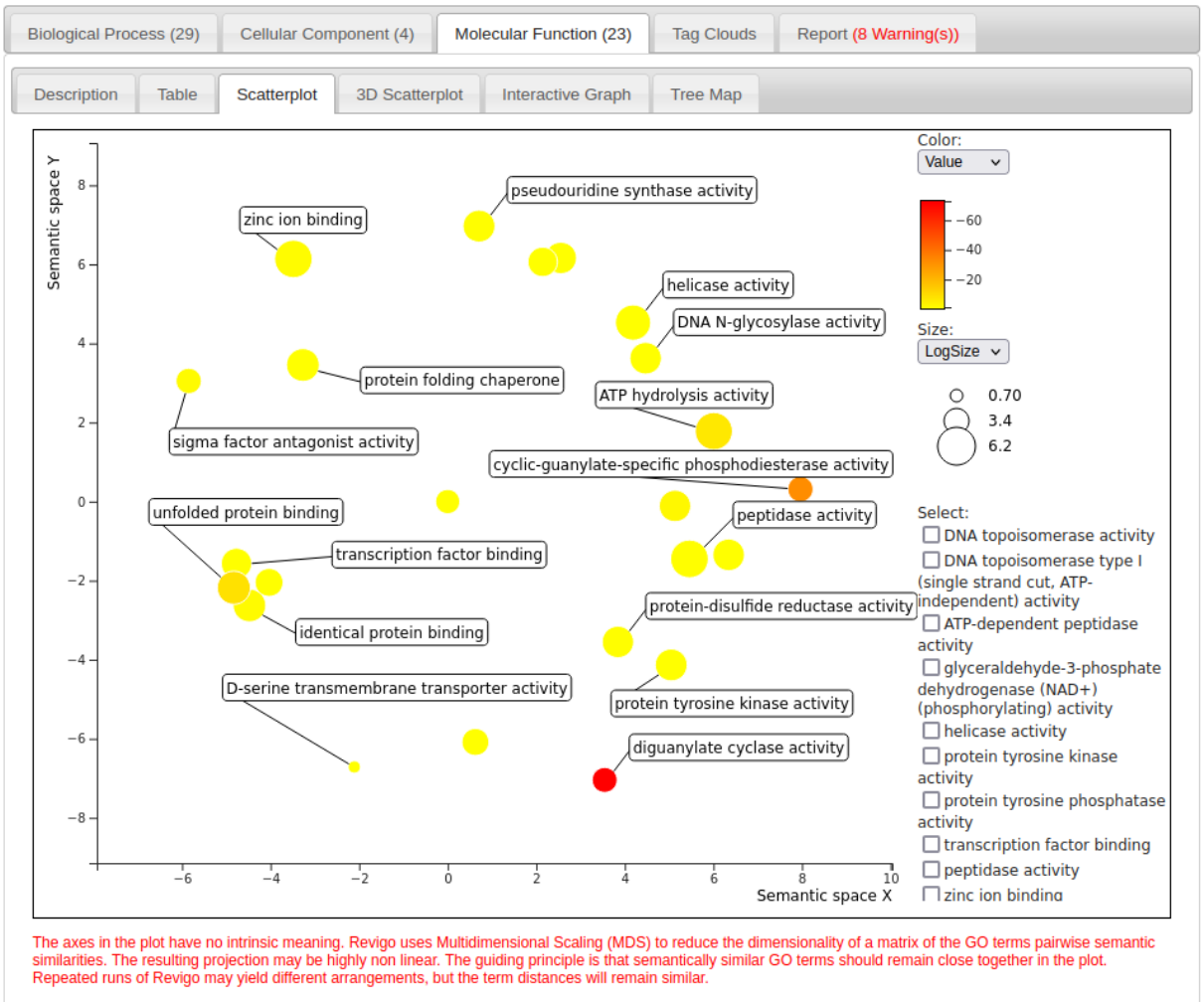


FIGURE 3.3 – Similarité sémantique des "GOTerm" - Fonctions moléculaires- χ^2 non ajustée - set 6 - REVIGO

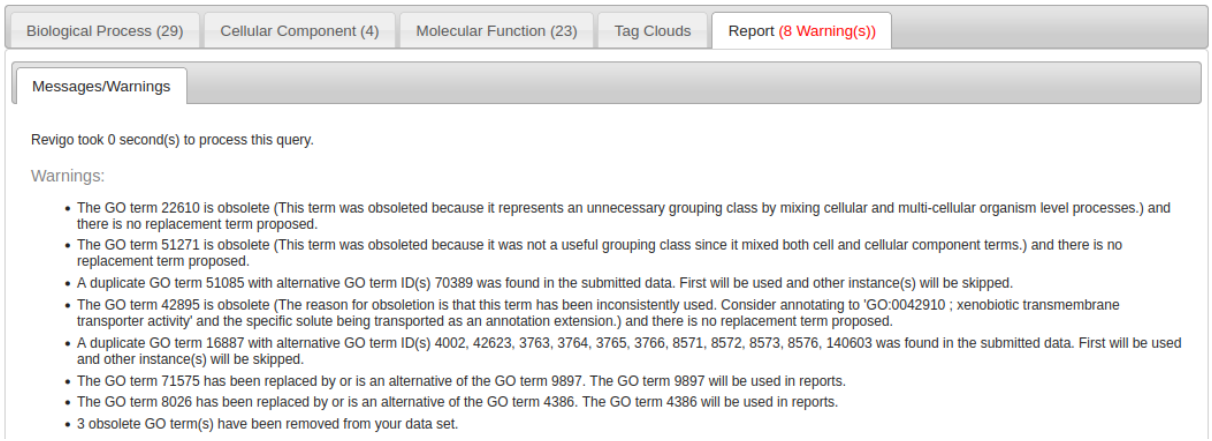


FIGURE 3.4 – Avertissement REVIGO χ^2 non ajustée - set 6 - REVIGO

CHAPITRE 4

INTERPRETATION

Il semblerait que l'ensemble de gènes requêtes 6 code pour des protéines impliquées dans une réponse au stress. Cette réponse inclue :

— la réception du signal (transduction sensorielle)

— la transmission du signal à la cellule (magnésium,transférase,protéines de liaison aux nucléotides,protéines chaperons,protéines de liaison à la GTP)

— et la réaction de défense (synthèse des exopolysaccharides, biosynthèse des Fimbrium) contre le stress occasionné.

La liste des termes de l'ontologie des gènes ("GOterm", base OBO) nous permet d'affiner l'hypothèse des fonctions biologiques des gènes inconnus. Il s'agirait de gènes impliquées dans une réponse à un stress thermique. La réponse au stress de la bactérie induit la création (ou le renforcement) du biofilm (processus de biosynthèse de l'acide colanique, activité phosphodiestérase spécifique du guanylate cyclique et diguanylate cyclase). La réponse au stress thermique semble inclure également :

— un changement biochimique de la membrane extérieure (biosynthèse de LPS, membrane externe bactérie Gram négative)

— une réduction de la mobilité (régulation négative de la motilité cellulaire dépendante du flagelle de type bactérien)

— renforcement de l'adhésion (création de firum)

— protection des protéines (via chaperonnes, transformation du repliement de protéines)

Il semblerait que la cellule bactérienne se prépare à affronter le stress en mettant en place une stratégie de défense plutôt que de fuite : perte de mobilité, changement de la structure de la membranne externe, renforcement du biofilm (bloc avec les bactéries voisines), création de fibrum qui renforce le phénomène d'adhésion (aux cellules eucaryotes de la paroi intestinal par exemple ?) protection des protéines. (*cf. figure 3.3*)

4.1 [v2] VALIDATION ET COHÉSION DE COEXPRESSION

Contrôle positif. Pour vérifier que la méthode retrouve bien une biologie connue, nous l'appliquons à un cas d'école : l'opéron *trp* (*trpA–E*, b1260–b1264), cinq gènes qui synthétisent le L-tryptophane. L'outil retourne en tête la voie `TRPSYN-PWY` (*L-tryptophan biosynthesis*, p = $2{,}1 \times 10^{-15}$, 5/5 gènes), suivie des superpathways des acides aminés aromatiques et du chorismate — exactement la hiérarchie attendue, puisque le tryptophane dérive du chorismate. L'outil ne fait pas que ■ tourner ■, il **retrouve la bonne réponse** sur un cas de référence.

Exploitation des données STRING. L'enrichissement indique *pour quelles* fonctions un ensemble de gènes est annoté. Une question complémentaire, indépendante : ces gènes forment-ils réellement un **module** — sont-ils plus coexprimés entre eux qu'un ensemble aléatoire de même taille ? Nous répondons par un test de permutation (2000 tirages) sur les arêtes de coexpression STRING, jusqu'ici seulement stockées dans la base.

Ensemble de gènes	gènes	force obs.	ratio vs aléatoire	p-value
Set d'étude (query 6)	113	305 975	**13,2**×	$\leq 5 \times 10^{-4}$
Opéron *trp* (contrôle)	5	9 934	**252**×	$\leq 5 \times 10^{-4}$

TABLE 4.1 – Cohésion de coexpression (STRING) — test de permutation. Le set d'étude est 13 fois plus coexprimé en interne que le hasard ; l'opéron *trp*, co-transcrit par définition, atteint 252 fois (contrôle positif).

Deux angles convergents. Nous disposons désormais de deux analyses *indépendantes* qui pointent vers la même conclusion : (1) l'**enrichissement** (annotations) montre que ces gènes codent la biosynthèse de l'acide colanique et le programme biofilm ; (2) la **coexpression** (STRING) montre qu'ils forment un module réel (13,2× au-dessus du hasard), et non une liste arbitraire. La convergence de deux méthodes indépendantes renforce la confiance dans l'interprétation : le set 6 est un module fonctionnel cohérent impliqué dans la réponse au stress et la formation du biofilm.

CHAPITRE 5

CONCLUSION

5.1 BIOLOGIQUE

L'ensemble de gènes inconnus 6 semble codées pour des protéines impliquée dans la réponse à un stress thermique. Ils sont impliqués depuis la réception du signal (de stress) à la cascade de réaction biochimique et cellulaire qui en découle.

Cette approche bio-informatique, via la récolte des informations importantes pour un projet, leur organisation en BD sous forme de graphe (Avec Neo4j par exemple), et leur interrogation automatique via un programme (python) est une formidable puissance pour analyser et caractériser une liste de gènes inconnus. Grâce à cette méthode, nous pouvons trouvé extrêmement facilement les fonctions biologiques et les publications associées. Par exemple, le travail de recherche de publication peut être très long et fastidieux. Cette méthode, permet vraiment de regrouper en un seul endroit, toutes informations relatives à un projet.
L'enjeu est de bien extraire les informations pertinentes (par exemple, il peut y avoir beaucoup de publication mentionnant chaque gène impliqué). Il s'agit d'une problématique classique dans bien des domaines, comment extraire le signal du bruit.

Il est important de fixer des limites raisonnables. Par exemple, à quel point est-il intéressant de poursuivre une étude qui possède un taux de recouvrement faible ?

5.2 RETOUR CRITIQUES

Nous n'avons pas intégré les "Alias" dans la base de donnée(nous nous en sommes rendue compte assez tardivement).
En comparant avec votre application (sur le site de silico.biotoul), nous ne trouvons pas les même résultats qu'avec la notre, qui semble être beaucoup moins restrictive. Néanmoins, les fonctions découvertes ne sont pas diamétralement opposées.

5.3 AMÉLIORATION

5.3.1 Application

- L'application mérite une optimisation de la sortie. Il manquerait des graphiques de description des données, distributions (histogrammes, boîte à moustaches) par exemple
- Nous pourrions ajouté d'autres organismes afin de voir si les gènes sont conservés ou non
- faire une analyse multivarié (ACP,regroupement des gènes, analyse discriminante), explorer les gènes co-exprimés
- Nous avons vu l'approche par ensemble, mais nous pourrions utiliser une autre approche, basée sur des matrices de similarité (distance) et détecter des schémas de co-présence/co-exclusion [1]
- **Population de référence (■ background ■) paramétrable.** Ici l'enrichissement est testé contre le génome entier (4315 gènes). Si l'ensemble requête provient d'une expérience ne couvrant qu'une partie des gènes (puce à ADN, RNA-seq), le background correct n'est pas le génome entier mais la liste des gènes réellement mesurés : les gènes non mesurés n'avaient aucune chance d'apparaître, et les inclure fausse les p-valeurs. Une option permettant de choisir le background (un sous-ensemble du génome plutôt que la totalité) rendrait l'outil directement applicable à des données de puce ou de RNA-seq.

1. cf. partie 2 et 3 de https://silico.biotoul.fr/enseignement/m2bbs/gdns-ap/GDNS-APG.Mise.en.oeuvre.html

5.3.2 Tests statistiques

Le test de Fischer exact et le test du χ^2 d'indépendance sont très souvent utilisées pour déceler une relation entre deux variables qualitative/binaire. Il s'agit de profils de présence/absence, c'est-à-dire des données binaires où chaque élément (gène, caractéristique, terme) est soit présent (1) ou absent (0). Néanmoins, il existe d'autres mesures de similarité que nous pourrions utiliser, comme par exemple :

- **le test de McNemar :** Ce test est utilisé pour comparer des fréquences entre deux échantillons appariés. Le coefficient Q de Yule et les odd-ratio sont aussi des mesures d'associations entre données binaires (Tan et al., 2004).
- **le test de Kruskal-Wallis :** test non paramétrique permettant de comparer plus de 2 échantillons et qui est en fait la généralisation du test de Mann-Whitney, mesure de l'association entre deux variables qualitatives
- **Coefficient de Jaccard :** couramment utilisé pour mesurer la similarité entre des ensembles binaires. Il est particulièrement adapté aux profils de présence/absence. Il mesure la proportion d'éléments en commun entre deux ensembles par rapport à la taille totale de leurs éléments
- **Distance de Hamming :** calcule le nombre de positions à partir desquelles deux profils binaires diffèrent. Plus les profils diffèrent, plus la distance de Hamming est élevée.
- **Coefficient de Dice :** similaire au coefficient de Jaccard et est également utilisé pour mesurer la similarité entre deux ensembles binaires.

Le coefficient de Jaccard est souvent utilisé pour des ensembles de grande taille, tandis que la distance de Hamming peut être appropriée si vous souhaitez mesurer la différence exacte entre deux profils.

Il est recommandé de tester plusieurs mesures de similarité et de choisir celle qui semble la plus pertinente pour chaque analyse en fonction des besoins et de la nature des données.

J'aurai peut être vu un test d'adéquation du χ^2 pour suivre la logique des 2 tests que nous avons réalisé ('binomial' et 'hypergeometric'). De plus, peut être que nous pourrions tester une autre stratégie pour ajuster les p-valeurs si multi-test (FDR).

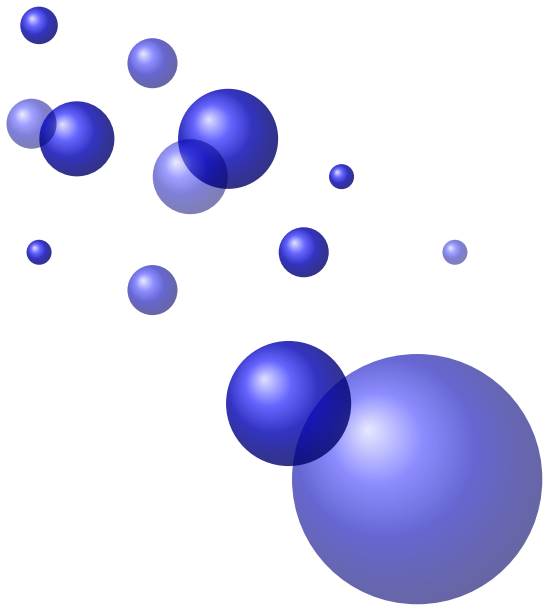
5.4 BILAN PERSONNEL

J'ai vraiment adoré ce projet, notamment les aspects suivant :

- comment collecter et où aller chercher les informations importantes
- avoir été sensibilisée aux problèmes d'identifiants, de références différents selon les sources de données et comment faire le lien entre eux
- comment créer une base de données interactive
- comment relier les informations entre elles, et avec des requêtes pouvoir répondre à des questions où il aurait fallu des heures si fait manuellement
- en connectant ainsi les données, voir des connexions entre les événements pas si évident au premier abort

J'aime beaucoup le principe d'organiser les données en graphe, plus en adéquation avec ma façon de voir les choses, plutôt que linéairement. J'ai entrevu des répercussions dans les futurs projets en biologie notamment, et j'ai hâte de réappliquer cette méthode à d'autres problématiques.
J'ai aimé programmer l'application python qui permet d'automatiser des demandes classiques à partir d'une liste de gènes d'intérêts. Super exemple de ce que nous pouvons faire avec python.

Même si la mise en place est longue, le gain de temps sur le long terme est indéniable.
Je suis très reconnaissante à mes camarades de classe Margaux DORE, Han PHAN et Claire D. pour l'étroite entraide entre nous, complètement indispensable. Nous avons continué Claire et moi de créer la base de donnée et l'application ensemble. Ce projet était très long, et au premier abord complexe, car beaucoup de notions à intégrer d'un coup. Le fait d'échanger et de travailler en équipe (équilibrée et dans la bonne humeur) avec Claire, nous a permis clairement (sans jeu de mot) de mieux comprendre l'ensemble. C'est définitivement un projet à faire à plusieurs ! La difficulté pour moi a été de savoir quand s'arrêter ! Beaucoup de questions, de choses à tester, et j'ai une certaine tendance à me laisser entraîner par ma curiosité. J'ai eu l'impression juste d'à peine ouvrir une porte sur de vastes possibilités.
Bilan : projet top, mais longgggg ! ! A proposer à faire en groupe, du moins pour la partie construction BD et application, je pense :)
Merci pour tout !



CHAPITRE 6

ANNEXE - SCRIPT R1 - TRANSFORMATION DONNÉES

TP01_graph

2023-10-11

```
library(tidyverse)
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✔ dplyr     1.1.2     ✔ readr     2.1.4
## ✔ forcats   1.0.0     ✔ stringr   1.5.0
## ✔ ggplot2   3.4.3     ✔ tibble    3.2.1
## ✔ lubridate 1.9.2     ✔ tidyr     1.3.0
## ✔ purrr     1.0.2
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(jsonlite)
```

```
##
## Attaching package: 'jsonlite'
##
## The following object is masked from 'package:purrr':
##
##     flatten
```

1.1.1.1 Choix des identifiants de référence 5keyword uniprot ou goterm)

```
uniprot <- read_tsv("../downloads/uniprotkb_proteome_UP000000625_2023_10_11.tsv.gz", show_col_types = F)
uniprot %>% head
```

Nombre de lignes attendu : 4403

```
dim(uniprot) # 4403 ok
```

```
## [1] 4403   10
```

A partir de ce tibble, il s'agit d'extraire les bnumbers de la colonne Gene names (ordered locus). La fonction str_extract permet d'extraire un pattern à partir d'une expression régulière (ici, ce sera 'b\d+') d'une chaîne de caractères.

```
mapping <- uniprot %>%
  select(Entry, names = `Gene Names`) %>%
  mutate(bnumber=str_extract(names, 'b\\d+')) %>%
  select(bnumber, uniprotID=Entry) %>%
  filter(!is.na(bnumber)) %>% # 2023 → P0DQD7 and P0A6D5 are lost (no bnumber)
  arrange(bnumber)
mapping %>% head
```

```
dim(mapping)
```

```
## [1] 4401    2
```

utile pour recup les identifiants

1.1.1.2 Reformatage des données

A présent, il s'agit de générer la liste des protéines associées à chaque mot-clé (colonne Keywords). Pour cela, on ne gardera que les colonnes Entry et Keywords, une jointure avec mapping va permettre de faire le mapping avec les bnumbers. La fonction separate_rows va nous servir à découper la colonne Keywords et répartir les mots-clés sur des lignes du tibble

```
keywords = uniprot %>%
  select(uniprotID=Entry, keyword=Keywords) %>%
  right_join(mapping, by="uniprotID") %>% # right join to remove those without bnumber
  separate_rows(keyword, sep=';') %>%
  select(bnumber, keyword) %>% #groupby par le mem mot clef
  arrange(bnumber)
keywords %>% head
```

```
dim(keywords)
```

```
## [1] 27506     2
```

Nombre de lignes attendu : 27506

Maintenant, il ne s'agit plus que de regrouper les bnumbers par mot-clé :

on prend que les 2 col qui nous intererss et a on veut a chaque bnumer recup tous les mots clef et divise col keyword pour associe a tous les bnumber

```
ref_sets = keywords %>%
  group_by(keyword) %>%
  summarise(count=n(), elements = list(bnumber)) %>%
  ungroup %>%
  filter(count>1)  %>%
  select(id=keyword, desc=count, elements)
ref_sets
```

Pour utiliser "facilement" ces données avec python, nous allons sauvegarder le tibble au format JSON avec la library library(jsonlite) :

///# output : /## Attaching package: 'jsonlite' /## The following object is masked from 'package:purrr': /## flatten

```
# exportation
ref_sets %>%
  toJSON %>%
  write("../ref.sets/uniprot.keywords.sets.json")
```

Aperçu du contenu :

```
jq . "../ref.sets/uniprot.keywords.sets.json" | head -15
```

```
## [
##   {
##     "id": "2Fe-2S",
##     "desc": 26,
##     "elements": [
##       "b0286",
##       "b0724",
##       "b0775",
##       "b0872",
##       "b0873",
##       "b1392",
##       "b1654",
##       "b1684",
##       "b1802",
##       "b1803",
```

InterPro

refaire comme pour avec keywords pour interpro ;

```
interpro = uniprot %>%
  select(uniprotID = Entry, keyword = InterPro) %>%
  right_join(mapping, by = "uniprotID") %>%
  separate_rows(keyword, sep = ';') %>%
  filter(!keyword=='') %>% # si on regarde la col interpro, vide a la fin. donc va etre aussi enregistre comme ''
vide !!! il faut le virer sinon pourri script python
  select(bnumber, keyword) %>%
  arrange(bnumber)

interpro %>% head
```

```
dim(interpro) #20037 sans vide 15k
```

```
## [1] 15636     2
```

Maintenant, il ne s'agit plus que de regrouper les bnumbers par mot-clé :

on prend que les 2 col qui nous intererss et a on veut a chaque bnumer recup tous les mots clef et divise col keyword pour associe a tous les bnumber

```
ref_sets_inter = interpro %>%
  group_by(keyword) %>%
  summarise(count=n(), elements = list(bnumber)) %>%
  ungroup %>%
  filter(count>1)  %>%
  select(id=keyword, desc=count, elements)
ref_sets_inter
```

Pour utiliser ces données avec python, nous allons sauvegarder le tibble au format JSON avec la library library(jsonlite) :

```
# exportation
ref_sets_inter %>%
  toJSON %>%
  write("../ref.sets/uniprot.interpro.sets.json")
```

Aperçu du contenu :

```
jq . "../ref.sets/uniprot.interpro.sets.json" | head -15
```

```
## [
##   {
##     "id": "IPR000014",
##     "desc": 14,
##     "elements": [
##       "b0400",
##       "b0619",
##       "b1201",
##       "b1285",
##       "b1323",
##       "b1341",
##       "b1489",
##       "b2067",
##       "b2219",
##       "b2869",
```

GO

refaire comme pour avec keywords pour Go ;

```
names(uniprot)
```

```
##  [1] "Entry"             "Reviewed"          "Entry Name"
##  [4] "Protein names"     "Gene Names"        "Organism"
##  [7] "Length"            "Gene Ontology IDs" "InterPro"
## [10] "Keywords"
```

```
go = uniprot %>%
  select(uniprotID = Entry, keyword = 'Gene Ontology IDs') %>%
  right_join(mapping, by = "uniprotID") %>%
  separate_rows(keyword, sep = '; ') %>%
  filter(!keyword=='') %>% # si on regarde la col interpro, vide a la fin. donc va etre aussi enregistre comme ''
vide !!! il faut le virer sinon pourri script python

  select(bnumber, keyword) %>%
  arrange(bnumber)

go %>% head
```

```
dim(go) #22874
```

```
## [1] 22430     2
```

Maintenant, il ne s’agit plus que de regrouper les bnumbers par mot-clé :

on prend que les 2 col qui nous intererss et a on veut a chaque bnumer recup tous les mots clef et divise col keyword pour associe a tous les bnumber

```
ref_sets_go = go %>%
  group_by(keyword) %>%
  summarise(count=n(), elements = list(bnumber)) %>%
  ungroup %>%
  filter(count>1)  %>%
  select(id=keyword, desc=count, elements)
ref_sets_go
```

Pour utiliser “facilement” ces données avec python, nous allons sauvegarder le tibble au format JSON avec la library library(jsonlite) :

///# output : /## Attaching package: ‘jsonlite’ /## The following object is masked from ‘package:purrr’: /## flatten

```
# exportation
ref_sets_go %>%
  toJSON %>%
  write("../ref.sets/uniprot.go.sets.json")
```

Aperçu du contenu :

```
jq . "../ref.sets/uniprot.go.sets.json" | head -15
```

```
## [
##   {
##     "id": "GO:0000014",
##     "desc": 4,
##     "elements": [
##       "b0228",
##       "b0397",
##       "b0398",
##       "b2761"
##     ]
##   },
##   {
##     "id": "GO:0000018",
##     "desc": 4,
##     "elements": [
```

TU

pour faire TU: 1/ charger all tu.txt -> vire Object ID, separer la colonne ‘Genes of transcription unit’ 2/ charger all genes -> recup bnumber (correspondence avec la liste de geneTU; et faire jonction 3/ exporter ## dL TU

```
TU <- read_tsv("../downloads/all-transcription-units.txt", show_col_types = F)
TU %>% head
```

```
# verifier la diff entre object id et Site
(TU$Site == TU$`Object ID`) %>% as.numeric() %>% sum(na.rm=TRUE)
```

```
## [1] 3690
```

```
which(is.na(TU$Site))
```

```
## [1]  336 1085 1923 1950 2055 2072 2810
```

```
which(is.na(TU$`Object ID`))
```

```
## [1]  336 1085 1923 1950 2055 2072 2810
```

```
# qui sont les de nan ?
TU %>% filter(is.na(Site))
```

```
names(TU)
```

```
## [1] "Site"                         "Left"
## [3] "Right"                        "Strand"
## [5] "Genes of transcription unit" "Object ID"
```

```
# check unque
dim(TU) #3697
```

```
## [1] 3697    6
```

```
length(unique(TU$`Object ID`)) #3691 or  3697 ! DONC DOUBLON
```

```
## [1] 3691
```

```
length(unique(TU$Site)) #3691
```

```
## [1] 3691
```

SEPARE LES GENETU, UNE LIGNE PAR gene TU

```
# Separation des nom des genes
TU_reshaped = TU %>% filter(!is.na(Site)) %>%
  select(ecocyID = Site,
         gene = 'Genes of transcription unit',
         Right = "Right",
         Left = "Left",
         Strand = "Strand" ) %>%
  separate_rows(gene, sep = ' // ') %>%
  filter(!gene=='') # si on regarde la col interpro, vide a la fin. donc va etre aussi enregistre comme '' vide !
!! il faut le virer sinon pourri script python

TU_reshaped %>% head
```

```
dim(TU_reshaped) #6586
```

```
## [1] 6586    5
```

```
# Check na
TU_reshaped %>% filter(is.na(ecocyID)) #%>% length
```

```
length(unique(TU_reshaped$gene)) # 4493
```

```
## [1] 4493
```

```
length(unique(TU_reshaped$ecocyID)) # 3690
```

```
## [1] 3690
```

```
# Verifie les doublons
doublons = TU_reshaped [duplicated(TU_reshaped),]
# Vérifiez si des doublons existent
if (nrow(doublons) > 0) {
  cat("Il y a des doublons dans le dataframe.\n")
  print(doublons)
} else {
  cat("Aucun doublon n'a été trouvé dans le dataframe.\n")
}
```

```
## Aucun doublon n'a été trouvé dans le dataframe.
```

Recherche correcpondance dans liste gene number

```
all_genes <- read_tsv("../downloads/All-instances-of-Genes-in-Escherichia-coli-K-12-substr.-MG1655.txt", show_col
_types = F)
all_genes %>% head
```

```
# Separe Names et recup b avec
all_genes_reshaped <- all_genes %>%
  select(ecocyID = 'Transcription Units', gene = Genes, extracted_terms = Names) %>%
  separate_rows(extracted_terms, sep = ' // ') %>%
  filter(!gene == '') %>%
  mutate(bnumber = str_extract(extracted_terms, "b\\d+")) %>%
  filter(!is.na(bnumber)) %>%
  select(-extracted_terms) %>%
  separate_rows(ecocyID, sep = ' // ')
# si on regarde la col interpro, vide a la fin. donc va etre aussi enregistre comme '' vide !!! il faut le virer
sinon pourri script python
all_genes_reshaped %>% head
```

```
dim(all_genes_reshaped %>% unique) #6665
```

```
## [1] 6665    3
```

```
# check na
all_genes_reshaped %>% filter(is.na(bnumber))
```

```
length(unique(all_genes_reshaped$gene)) # 4489 (tu 4493)
```

```
## [1] 4489
```

```
length(unique(all_genes_reshaped$ecocyID)) # 3635 (tu 3690)
```

```
## [1] 3635
```

```
length(unique(all_genes_reshaped$bnumber)) # 4548
```

```
## [1] 4548
```

creer file

```
# Jointure entre les 2 tables, TU et all genes sur geneTU,

tus_to_export <- all_genes_reshaped %>%
  left_join(TU_reshaped, by = c("gene","ecocyID")) %>% filter(!is.na(bnumber)) %>% mutate(desc = paste(Strand, Le
ft, Right, sep = " , "))
# car doublons; mais la combinaison des 2 est unique !!!!
tus_to_export %>% head()
```

```
dim(tus_to_export) #6666 avec na 6528 sans na et 6665 dans all gene
```

```
## [1] 6666    7
```

```
# Check na
tus_to_export %>% filter(is.na(bnumber)) #%>% length
```

Pour utiliser ces données avec python, nous allons sauvegarder le tibble au format JSON avec la library library(jsonlite) :

///# output : /## Attaching package: ‘jsonlite’ /## The following object is masked from ‘package:purrr’: /## flatten

```
# exportation
tus_to_export %>%
  toJSON %>%
  write("../ref.sets/ecocys.tu.sets.json")
```

Aperçu du contenu :

```
jq . "../ref.sets/ecocys.tu.sets.json" | head -15
```

```
## [
##   {
##     "ecocyID": "TU00512",
##     "gene": "G0-16636",
##     "bnumber": "b4759",
##     "Right": 1992117,
##     "Left": 1992180,
##     "Strand": "-",
##     "desc": "- , 1992180 , 1992117"
##   },
##   {
##     "ecocyID": "TU0-13901",
##     "gene": "G7686",
##     "bnumber": "b3241",
##     "Right": 3389337,
```

EXPORT EN CSV pour neo4j

```
# pour les noeuds
tus_to_export %>% unique %>% write_csv("../neo4j.import/ecocyc.tu.csv")


# pour les relations connextion avec gene
tus_to_export %>% select(c(ecocyID,bnumber)) %>% unique %>% write_csv("../neo4j.import/ecocyc.tu.genes.csv")
```

Pathway

pour faire pathway: 1/ charger pathway.txt -> vire Object ID, separer la colonne ‘Genes of pathway’ utiliser all genes pour correspondances -> recup bnumber (correspondence avec la liste de geneTU; et faire jonction 3/ exporter

dL pathway

```
pathway <- read_tsv("../downloads/All-instances-of-Pathways-in-Escherichia-coli-K-12-substr.-MG1655.txt", show_co
l_types = F)
pathway %>% head
```

quick explore

```
# qui sont les de nan ?
pathway %>% filter(is.na(Pathways))
```

```
names(pathway)
```

```
## [1] "Pathways"         "Genes of pathway" "Object ID"        "Common-Name"
```

-> utilisation variables common names pour desc

```
# check unque
dim(pathway) # 438
```

```
## [1] 438   4
```

SEPARE LES GENES implique pathway, UNE LIGNE PAR gene pathway

```
# Separation des nom des genes, filter(!is.na(Site)) %>% pas necessaires
pathway_reshaped = pathway %>%
  select(ecocyID = Pathways,
         gene = "Genes of pathway",
         desc = "Common-Name") %>%
  separate_rows(gene, sep = ' // ') %>%
  filter(!gene=='') # enregistre le vide a la fin. donc va etre aussi enregistre comme '' vide !!! il faut le vir
er sinon pourri script python

pathway_reshaped %>% head
```

```
dim(pathway_reshaped) #2849
```

```
## [1] 2849    3
```

```
# Check na
pathway_reshaped %>% filter(is.na(ecocyID)) #ok
```

```
# check unique
length(unique(pathway_reshaped$gene)) # 1156
```

```
## [1] 1156
```

```
length(unique(pathway_reshaped$ecocyID)) # 438
```

```
## [1] 438
```

```
# Verifie les doublons
doublons2 <- pathway_reshaped[duplicated(pathway_reshaped[c("ecocyID", "gene")]), ] # Vérifier si des doublons ex
istent
if (nrow(doublons2) > 0) {
  cat("Il y a des doublons dans le dataframe.\n")
  print(doublons2)
} else {
  cat("Aucun doublon n'a été trouvé dans le dataframe.\n")
}
```

```
## Aucun doublon n'a été trouvé dans le dataframe.
```

une ligne = un couple pathway/gene

Recherche correcpondance dans liste gene number dans all gene (bd ecocyd inclu)

j ai deja recupere les bnumber a partir de all_genes

```
all_genes%>% head
```

```
 all_genes_reshaped %>% head
```

```
dim(all_genes) #4523
```

```
## [1] 4523    6
```

```
dim(all_genes_reshaped) #6666
```

```
## [1] 6666    3
```

```
dim(all_genes_reshaped %>% unique) #6665
```

```
## [1] 6665    3
```

creer fichiers

```
# Jointure entre les 2 tables, pathway et all genes. je veux recuperer la colonne bnumber

pathways_to_export = left_join(pathway_reshaped, all_genes_reshaped %>% select(gene, bnumber), by = "gene")  %>%
filter(!is.na(bnumber)) %>%
  distinct()
```

```
## Warning in left_join(pathway_reshaped, all_genes_reshaped %>% select(gene, : Detected an unexpected many-to-ma
ny relationship between `x` and `y`.
## i Row 1 of `x` matches multiple rows in `y`.
## i Row 43 of `y` matches multiple rows in `x`.
## i If a many-to-many relationship is expected, set `relationship =
##   "many-to-many"` to silence this warning.
```

```
# car doublons; mais la combinaison des 2 est unique !!!!
pathways_to_export %>% head()
```

```
# Check duplicated
dim(pathways_to_export) # 4936 avec na /4932 sans na et 6665 dans all gene
```

```
## [1] 2863    4
```

```
dim(pathways_to_export %>% unique) #2863
```

```
## [1] 2863    4
```

```
length(unique(pathways_to_export$bnumber))
```

```
## [1] 1165
```

expected 2849 (cf patways dimension). need remove duplicates -> 2863

```
# Check the difference between the 2 (why 2849 and 2863 row ). Duplicates?
doublons22 <- pathways_to_export[duplicated(pathways_to_export[c("ecocyID", "gene")]), ] # Vérifier si des doublo
ns existent
if (nrow(doublons22) > 0) {
  cat("Il y a des doublons dans le dataframe.\n")
  print(doublons22)
} else {
  cat("Aucun doublon n'a été trouvé dans le dataframe.\n")
}
```

```
## Il y a des doublons dans le dataframe.
## # A tibble: 18 × 4
##    ecocyID             gene    desc                                    bnumber
##    <chr>               <chr>   <chr>                                   <chr>
##  1 GALACTCAT-PWY       EG20049 D-galactonate degradation               b4477
##  2 GALACTCAT-PWY       GB4478  D-galactonate degradation               b4478
##  3 ECASYN-PWY          G7800   enterobacterial common antigen biosynthe… b4405
##  4 ECASYN-PWY          G7800   enterobacterial common antigen biosynthe… b4481
##  5 PWY0-521            G7724   fructoselysine and psicoselysine degrada… b3372
##  6 PWY0-521            G7724   fructoselysine and psicoselysine degrada… b4474
##  7 GLYCOLATEMET-PWY    G7544   glycolate and glyoxylate degradation I    b4468
##  8 GLYCOLATEMET-PWY    G0-8601 glycolate and glyoxylate degradation I    b4467
##  9 GLYOXDEG-PWY        G7544   glycolate and glyoxylate degradation II   b4468
## 10 GLYOXDEG-PWY        G0-8601 glycolate and glyoxylate degradation II   b4467
## 11 LCYSDEG-PWY         G7622   L-cysteine degradation II                 b3108
## 12 LCYSDEG-PWY         G7622   L-cysteine degradation II                 b4470
## 13 SERDEG-PWY          G7624   L-serine degradation                      b3112
## 14 SERDEG-PWY          G7624   L-serine degradation                      b4471
## 15 PWY-8227            G7800   superpathway of enterobacterial common a… b4405
## 16 PWY-8227            G7800   superpathway of enterobacterial common a… b4481
## 17 GLYCOL-GLYOXDEG-PWY G7544   superpathway of glycol metabolism and de… b4468
## 18 GLYCOL-GLYOXDEG-PWY G0-8601 superpathway of glycol metabolism and de… b4467
```

It seems that, if “ecocyID” and “gene” are similar, the bnumber is not, for example: PWY0-521 and G7724, b3372 and b4474
Sometimes, bnumber and gene number are the same (b4467,G0-8601), but not the pathway. We need to check on 3 conditions: gene, bnumber, ecocyID :

```
#  Check duplicates. need remove if the bnumber, gene and ecocyID are the same
doublons22 <- pathways_to_export[duplicated(pathways_to_export[c("ecocyID", "gene","bnumber")]), ] # Vérifier si
des doublons existent
if (nrow(doublons22) > 0) {
  cat("Il y a des doublons dans le dataframe.\n")
  print(doublons22)
} else {
  cat("Aucun doublon n'a été trouvé dans le dataframe.\n")
}
```

```
## Aucun doublon n'a été trouvé dans le dataframe.
```

OK; we need to keep everything.

```
# Check na
pathways_to_export %>% filter(is.na(bnumber)) # ok (%>% length)
```

```
# Check if all the row from pathway_reshaped are in the df pathways_to_export
diff_rows <- anti_join(pathway_reshaped, pathways_to_export, by = "gene")

# Print differences
print(diff_rows)
```

```
## # A tibble: 4 × 3
##   ecocyID                   gene  desc
##   <chr>                     <chr> <chr>
## 1 ILEUSYN-PWY               G8221 L-isoleucine biosynthesis I (from threonine)
## 2 VALSYN-PWY                G8221 L-valine biosynthesis
## 3 BRANCHED-CHAIN-AA-SYN-PWY G8221 superpathway of branched chain amino acid bio…
## 4 THREOCAT-PWY              G8221 superpathway of L-threonine metabolism
```

```
# Check gene G8221
all_genes_reshaped %>% filter(gene == "G8221") #none
```

```
# so check if the corresponding ecocyID exist
all_genes_reshaped %>% filter('ecocyID' == "ILEUSYN-PWY")
```

The Genes ‘G8221’ is not present in all_genes df where the bnumber is.

Pour utiliser ces données avec python, nous allons sauvegarder le tibble au format JSON avec la library library(jsonlite) :

///# output : /## Attaching package: ‘jsonlite’ /## The following object is masked from ‘package:purrr’: /## flatten

```
# exportation
pathways_to_export %>%
  toJSON %>%
  write("../ref.sets/ecocys.pathways.sets.json")
```

Aperçu du contenu :

```
jq . "../ref.sets/ecocys.pathways.sets.json" | head -15
```

```
## [
##   {
##     "ecocyID": "PWY0-862",
##     "gene": "EG11528",
##     "desc": "(5Z)-dodecenoate biosynthesis I",
##     "bnumber": "b1288"
##   },
##   {
##     "ecocyID": "PWY0-862",
##     "gene": "EG11318",
##     "desc": "(5Z)-dodecenoate biosynthesis I",
##     "bnumber": "b1093"
##   },
##   {
##     "ecocyID": "PWY0-862",
```

EXPORT EN CSV pour neo4j

```
# pour les noeuds
pathways_to_export %>% unique %>% write_csv("../neo4j.import/ecocyc.pathways.csv")


# pour les relations connexion avec gene
pathways_to_export %>% select(c(ecocyID,bnumber)) %>% unique %>% write_csv("../neo4j.import/ecocyc.pathways.genes
.csv")
```

CHAPITRE 7

ANNEXE - SCRIPT R2 - INTEGRATION DES FICHIERS CSV DANS LA BASE DE DONNÉES NEO4J

R Notebook

Code

This is an R Markdown (http://rmarkdown.rstudio.com) Notebook. When you execute code within the notebook, the results appear beneath the code.

Try executing this chunk by clicking the *Run* button within the chunk or by placing your cursor inside it and pressing *Ctrl+Shift+Enter*.

charge library

if (!require('neo4r')) { # client neo4j install.packages('neo4r') }

```
library(neo4r)
neodb = neo4j_api$new(
  url = "http://localhost:7474",
  user = "neo4j",
  password = "omybioinfo"
)
cypher = function(query, neo4j=neodb, ...) call_neo4j(query=query, con=neo4j, ...)
```

1.3.2.1 Nettoyage

avant de creer la bd surtout pas apres...

```
'MATCH ()-[r]-() DELETE(r)' %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
'MATCH ()-[r]-() RETURN count(r)' %>% cypher %>% unlist
```

```
count(r).value
            0
```

```
# attention supprime tout
'MATCH (n) DELETE(n)' %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
'MATCH (n) RETURN count(n)' %>% cypher %>% unlist
```

```
count(n).value
            0
```

Genes 1.1.1.1 Choix des identifiants de référence, import genes

extraire position des gene sur ecoli

```
library(tidyverse)
library(jsonlite)
```

1ere chose a faire cest crere les noeuds

```
ncbi <- read_tsv("../data/mapping.bed.tsv", show_col_types = F)
ncbi %>% head
```

```
names(ncbi)
```

```
[1] "bnumber" "rank"    "strand"  "begin"
[5] "end"
```

Nombre de lignes attendu : 4403

```
dim(ncbi) # 4315 ok
```

```
[1] 4315    5
```

```
length(unique(ncbi$bnumber))
```

```
[1] 4305
```

Load CSV GENOME DE REF E.COLI (GENES)

```
# IMPORT

"LOAD CSV WITH HEADERS FROM 'file:///mapping.bed.tsv' AS row FIELDTERMINATOR '\t'
CREATE (n:Gene)
SET n = row,
 n.id = row.bnumber,
 n.organism = toInteger('511145'),
 n.rank = toInteger(row.rank),
 n.strand = row.strand,
 n.begin = toInteger(row.begin),
 n.end = toInteger(row.end)
 " %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

verif

```
'MATCH (n:Gene) RETURN count(n)' %>% cypher %>% unlist #4315
```

```
count(n).value
        4315
```

VISU

```
"MATCH (n:Gene) RETURN n LIMIT 6" %>% cypher %>% .$n
```

Index sinon l’ajout de liens et les requêtes peuvent être très longs

```
'CREATE INDEX ON :Gene(id)' %>% cypher
```

```
$error_code
[1] "Neo.ClientError.Schema.EquivalentSchemaRuleAlreadyExists"

$error_message
[1] "An equivalent index already exists, 'Index( id=3, name='index_817871c2', type='GENERAL BTREE', schema=(:Gene {id}), indexProvider='native-btree-1.0' )'."
```

1.3.2.2.2 Keywords et liens Keyword → Gene (uniprot)

CHARGE NOEUD KEYWORDS

```
# CHECK
keywords %>% select(keyword) %>% unique
```

```
# Load CSV
keywords %>%
  select(keyword) %>%
  unique %>%
  write_csv("../neo4j.import/uniprot.keywords.csv")

'LOAD CSV WITH HEADERS FROM "file:///uniprot.keywords.csv" AS row
CREATE (n:Keyword)
SET n = row,
 n.id = row.keyword
'  %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
'CREATE INDEX ON :Keyword(id)' %>% cypher
```

```
$error_code
[1] "Neo.ClientError.Schema.EquivalentSchemaRuleAlreadyExists"

$error_message
[1] "An equivalent index already exists, 'Index( id=4, name='index_9193b500', type='GENERAL BTREE', schema=(:Key
word {id}), indexProvider='native-btree-1.0' )'."
```

VERIF

```
'MATCH (n:Keyword) RETURN count(n)' %>% cypher %>% unlist #385
```

```
count(n).value
          385
```

Liens Keyword → Gene

```
keywords %>% write_csv("../neo4j.import/uniprot.keywords.genes.csv")
```

```
"MATCH (:Keyword)-[r:describes]->(:Gene) DELETE r" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"
LOAD CSV WITH HEADERS FROM 'file:///uniprot.keywords.genes.csv' AS line
MATCH (k:Keyword),(g:Gene)
WHERE k.id=line.keyword AND g.id=line.bnumber
WITH k,g
MERGE (k)-[:describes]->(g)
" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

VERIF

```
"MATCH (:Keyword)-[r:describes]->(:Gene) RETURN count(r)" %>% cypher %>% unlist #27323
```

```
count(r).value
        27323
```

1.3.2.2.3 PubMed

Create files

./scripts/gbff.to.bnumber.GeneID.py downloads/GCF_000005845.2.gbff.gz > generated.data/mapping.bnumber_ncbi.tsv

```
pmids = read_tsv("../data/bnumber.PMID.tsv", col_types = "cc")
pmids %>% select(PMID) %>% unique %>% write_csv("../neo4j.import/ncbi.pmid.csv")
pmids %>% rename(gene_id = bnumber) %>% write_csv("../neo4j.import/ncbi.pmid.genes.csv")
```

Export dans Noe4j

```
# LOAD dans neo4j
"MATCH (:PubMed)-[r:cites]->(:Gene) DELETE r" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
'MATCH (n:PubMed) DELETE n' %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
'LOAD CSV WITH HEADERS FROM "file:///ncbi.pmid.csv" AS row
CREATE (n:PubMed)
SET n = row,
 n.id = row.PMID
'  %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
'CREATE INDEX ON :PubMed(id)' %>% cypher
```

```
$error_code
[1] "Neo.ClientError.Schema.EquivalentSchemaRuleAlreadyExists"

$error_message
[1] "An equivalent index already exists, 'Index( id=5, name='index_66c3ecb5', type='GENERAL BTREE', schema=(:Pub
Med {id}), indexProvider='native-btree-1.0' )'."
```

Vérification

```
'MATCH (n:PubMed) RETURN count(n)' %>% cypher %>% unlist #35320
```

```
count(n).value
        35320
```

Import du fichier CSV LIEN ARETE

```
"MATCH (:PubMed)-[r:cites]->(:Gene) DELETE r" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"
LOAD CSV WITH HEADERS FROM 'file:///ncbi.pmid.genes.csv' AS line
MATCH (p:PubMed),(g:Gene)
WHERE p.id=line.PMID AND g.id=line.gene_id
WITH p,g
MERGE (p)-[:cites]->(g)
" %>% cypher
```

No data returned.

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

Vérification

```
"MATCH (:PubMed)-[r:cites]->(:Gene) RETURN count(r)" %>% cypher %>% unlist #101841
```

```
count(r).value
        101841
```

Complétez cette partie afin de charger dans la base de données :

Les domaines InterPro -> Ok
les pathways,
les unités de transcription (TUs),

interpro

Load CSV NODES

```
interpro %>%
  select(keyword) %>%
  unique %>%
  write_csv("../neo4j.import/uniprot.interpro.csv")

# Check clean the bd
"MATCH (:InterPro)-[r]-() DELETE r" %>% cypher
```

No data returned.

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"MATCH (n:InterPro) DELETE n" %>% cypher
```

No data returned.

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
#nom de la BD
'LOAD CSV WITH HEADERS FROM "file:///uniprot.interpro.csv" AS row
CREATE (n:InterPro)
SET n = row,
 n.id = row.keyword
'  %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
# 6,760
```

verif

```
'MATCH (n:InterPro) RETURN count(n)' %>% cypher %>% unlist #6760 expected 6679
```

```
count(n).value
         6760
```

VERIF

```
"MATCH (n:InterPro) RETURN n LIMIT 6" %>% cypher %>% .$n
```

INDEX

```
'CREATE INDEX ON :InterPro' %>% cypher
```

```
$error_code
[1] "Neo.ClientError.Statement.SyntaxError"

$error_message
[1] "Invalid input ': expected \"(\" (line 1, column 26 (offset: 25))\n\"CREATE INDEX ON :InterPro\"\n                         ^
"
```

RECUP lien arete

```
interpro %>% write_csv("../neo4j.import/uniprot.interpro.genes.csv")
```

```
#LOAD DATA ON NEO4J
"MATCH (:InterPro)-[r:encoded_in]->(:Gene) DELETE r" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"
LOAD CSV WITH HEADERS FROM 'file:///uniprot.interpro.genes.csv' AS line
MATCH (k:InterPro),(g:Gene)
WHERE k.id=line.keyword AND g.id=line.bnumber
WITH k,g
MERGE (k)-[:encoded_in]->(g)
" %>% cypher
```

No data returned.

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"MATCH (n:InterPro)-[r:encoded_in]->(:Gene) RETURN count(r)" %>% cypher %>% unlist # 15401 15515
```

```
count(r).value
        15515
```

GO

Load CSV NODES

```
"MATCH (:GOTerm)-[r]-() DELETE r" %>% cypher
```

No data returned.

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"MATCH (n:GOTerm) DELETE n" %>% cypher
```

No data returned.

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"
LOAD CSV WITH HEADERS FROM 'file:///go.nodes.tsv' AS row FIELDTERMINATOR '\t'
CREATE (n:GOTerm)
SET n.id = row.id,
n.name = row.desc,
n.desc  = row.def,
n.namespace = row.namespace
"  %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

Verif

```
"MATCH (n:GOTerm) RETURN count(n)" %>% cypher %>% unlist # 42887
```

```
count(n).value
        42887
```

```
# 42887
```

```
"CREATE INDEX ON :GOTerm(id)" %>% cypher
```

```
$error_code
[1] "Neo.ClientError.Schema.EquivalentSchemaRuleAlreadyExists"

$error_message
[1] "An equivalent index already exists, 'Index( id=6, name='index_1a03f871', type='GENERAL BTREE', schema=(:GO
Term {id}), indexProvider='native-btree-1.0' )'."
```

expected /## $error_code /## [1] “Neo.ClientError.Schema.EquivalentSchemaRuleAlreadyExists” ## /## $error_message /## [1] “An equivalent index already exists, ‘Index(id=9, name=’index_1a03f871’, type=‘GENERAL BTREE’, schema=(:GOTerm {id}), indexProvider=‘native-btree-1.0’)’.” Extait du contenu :

```
"MATCH (n:GOTerm) RETURN n LIMIT 6" %>% cypher %>% .$n
```

```
Column 2 ['namespace'] of item 2 appears in position 1 in item 1. Set use.names=TRUE to match by column name, or
use.names=FALSE to ignore column names. use.names='check' (default from v1.12.2) emits this message and procee
ds as if use.names=FALSE for  backwards compatibility. See news item 5 in v1.12.2 for options to control this messag
e.
```

RECUP lien arete relation is a

```
"
LOAD CSV WITH HEADERS FROM 'file:///go.is_a.edges.tsv' AS line FIELDTERMINATOR '\t'
MATCH (t1:GOTerm),(t2:GOTerm)
WHERE t1.id=line.term1 AND t2.id=line.term2
WITH t1,t2
MERGE (t2)-[r:is_a]->(t1)
" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"MATCH ()-[r:is_a]->() RETURN count(r)" %>% cypher %>% unlist #68521
```

```
count(r).value
        68521
```

RECUP lien arete relation part of

```
"
LOAD CSV WITH HEADERS FROM 'file:///go.part_of.edges.tsv' AS line FIELDTERMINATOR '\t'
MATCH (t1:GOTerm),(t2:GOTerm)
WHERE t1.id=line.term1 AND t2.id=line.term2
WITH t1,t2
MERGE (t2)-[r:part_of]->(t1)
" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"MATCH ()-[r:part_of]->() RETURN count(r)" %>% cypher %>% unlist #6785
```

```
count(r).value
        6785
```

```
GOTerms = uniprot %>%
  select(uniprotID=Entry, GOTerm=`Gene Ontology IDs`) %>%
  right_join(mapping, by = join_by(uniprotID)) %>% # right join to remove those without bnumber
  separate_rows(GOTerm, sep='; ') %>%
  select(bnumber, GOTerm) %>%
  arrange(bnumber)
GOTerms
```

Save CSV file for Neo4J

```
GOTerms %>% write_csv("../neo4j.import/uniprot.GOTerm.bnumber.csv")
```

Create conexion Merge into graph

```
"
LOAD CSV WITH HEADERS FROM 'file:///uniprot.GOTerm.bnumber.csv' AS line
MATCH (t:GOTerm),(g:Gene)
WHERE t.id=line.GOTerm AND g.id=line.bnumber
WITH t,g
MERGE (t)-[:annotates]->(g)
" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

Vérification

```
"MATCH ()-[r:annotates]->() RETURN count(r)" %>% cypher %>% unlist #22373
```

```
count(r).value
        22373
```

TUs

Charge noeud TU (list unique TU/geneTU)-> 6665

```
names(tus_to_export)
```

```
[1] "ecocyID" "gene"    "bnumber" "Right"
[5] "Left"    "Strand"  "desc"
```

```
dim(tus_to_export)
```

```
[1] 6545    7
```

```
length(unique(tus_to_export$ecocyID))
```

```
[1] 3634
```

```
# LOAD dans neo4j liste noeud 5TU unique et toutes les info relatives a chaque Tu)
"MATCH (:TU)-[r:harbors]->(:Gene) DELETE r" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
'MATCH (n:TU) DELETE n' %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"LOAD CSV WITH HEADERS FROM 'file:///ecocyc.tu.csv' AS row
CREATE (n:TU)
SET n=row,
 n.id = row.ecocyID,
 n.strand = row.Strand,
 n.begin = toInteger(row.Left),
 n.end = toInteger(row.Right),
 n.desc  = row.desc,
 n.organism = toInteger('511145')
 " %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

verif

```
'MATCH (n:TU) RETURN count(n)' %>% cypher %>% unlist #3634 expected 3696
```

```
count(n).value
        3634
```

VISU

```
"MATCH (n:TU) RETURN n LIMIT 6" %>% cypher %>% .$n
```

Index sinon l’ajout de liens et les requêtes peuvent être très longs

```
'DROP INDEX index_1e15df2a'%>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
'CREATE INDEX ON :TU(id)' %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

Creation LIEN ARETE relation connection association

```
"MATCH (:TU)-[r:harbors]->(:Gene) DELETE r" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
# EXPORT
"
LOAD CSV WITH HEADERS FROM 'file:///ecocyc.tu.genes.csv' AS line
MATCH (p:TU),(g:Gene)
WHERE p.id=line.ecocyID AND g.id=line.bnumber
WITH p,g
MERGE (p)-[:harbors]->(g)
" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

Vérification

```
"MATCH (n:TU)-[r:harbors]->(:Gene) RETURN count(r)" %>% cypher %>% unlist # expected 6841
```

```
count(r).value
         6170
```

Pathway

Charge noeud pathways (list unique pathways/gene)-> 6665

```
names(pathways_to_export)
```

```
[1] "ecocyID" "gene"    "desc"    "bnumber"
```

Export to Neo4J

```
# LOAD dans neo4j
"MATCH (:Pathway)-[r:requires]->(:Gene) DELETE r" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
'MATCH (n:Pathway) DELETE n' %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"LOAD CSV WITH HEADERS FROM 'file:///ecocyc.pathways.csv' AS row
CREATE (n:Pathway)
SET n=row,
 n.id = row.ecocyID,
 n.organism = toInteger('511145'),
 n.desc  = row.desc
 " %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

verif

```
'MATCH (n:Pathway) RETURN count(n)' %>% cypher %>% unlist #438 ->OK expected 442
```

```
count(n).value
          438
```

VISU

```
"MATCH (n:Pathway) RETURN n LIMIT 15" %>% cypher %>% .$n
```

Index sinon l’ajout de liens et les requêtes peuvent être très longs

```
'CREATE INDEX ON :Pathway(id)' %>% cypher
```

```
$error_code
[1] "Neo.ClientError.Schema.EquivalentSchemaRuleAlreadyExists"

$error_message
[1] "An equivalent index already exists, 'Index( id=8, name='index_bec32f20', type='GENERAL BTREE', schema=(:Path
way {id}), indexProvider='native-btree-1.0' )'."
```

Creation LIEN ARETE relation connection association

```
"MATCH (:Pathway)-[r:requires]->(:Gene) DELETE r" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
# EXPORT
"
LOAD CSV WITH HEADERS FROM 'file:///ecocyc.pathways.genes.csv' AS line
MATCH (p:Pathway),(g:Gene)
WHERE p.id=line.ecocyID AND g.id=line.bnumber
WITH p,g
MERGE (p)-[:requires]->(g)
" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

Vérification

```
"MATCH (:Pathway)-[r:requires]->(:Gene) RETURN count(r)" %>% cypher %>% unlist #38194
```

```
count(r).value
        2747
```

CHAPITRE 8

ANNEXE - SCRIPT PYTHON - APPLICATION ENRICHISSEMENT

```
#!/bin/env python
# conda install python3 biopython requests
# ./gbff.to.bnumber.GeneID.py > mapping.bnumber_ncbi.tsv

# UPLOAD PACKAGES
from py2neo import * # conda install py2neo, monotonic
from py2neo.matching import *
import argparse
from pprint import pprint
from os.path import isfile
import json
from scipy.stats import binom, hypergeom, chi2_contingency  # pip install scipy


# CONNECTION TO NEO4J
neo = Graph("bolt://localhost:7687",
            auth=("neo4j","omybioinfo"))
nodes = NodeMatcher(neo)


# SCRIPT PARAMETERS ##########################################################
parser = argparse.ArgumentParser(description='Search enriched terms/categories
in the provided (gene) set')
parser.add_argument('-q', '--query', required=True,
help='Query set (gene nodes in Neo4j).')
parser.add_argument('-t', '--type', required=True, choices=['Keyword', 'Pathway','TU','GOTer
parser.add_argument('-a', '--alpha', required=False, type=float, default=0.05,
help='Significance threshold.')
parser.add_argument('-c', '--adjust', required=False, action="store_true",
help='Adjust for multiple testing (FDR).')
parser.add_argument('-m', '--measure', required=False, default='binomial',
help='Dissimilarity index: binomial (default), hypergeometric, chi2 or coverage')
parser.add_argument('-l', '--limit', required=False, type=int, default=0,
help='Maximum number of results to report.')
parser.add_argument('-v', '--verbose', required=False, action="store_true", help='Talk a lot
param = parser.parse_args()


# Path Query
path = '../query/query6.txt'


# LOAD QUERY #################################################################
text = param.query
query = set()
if isfile(text):
    with open(text) as f:
        content = ' '.join(f.read().split('\n')).split()
        query |= set(content)
```

```
else: # parse string
    query |= set(text.split())

if param.verbose:
  print(f'query set: {query}')

# LOAD TARGET/REFERENCE SETS ############################################################
population = nodes.match("Gene")
pprint(population)

# POPULATION SIZE = total number of genes in the database
n_gene_total = population.count() # len(population)
pprint(f"population size: {n_gene_total}")

# QUERY SIZE = number of genes in the query
query_size = len(query)
pprint(f"Number of genes in the query: {query_size}")

# EXTRACT query genes from the database:

target = param.type
if target == 'Keyword': # if we want to search for keywords
    cypher = f"MATCH (k:{target})-->(g:Gene) WHERE g.id IN {list(query)}
    RETURN DISTINCT k"
    res = neo.run(cypher)#.to_table()

elif target == 'GOTerm': # EXTRACT GO TERMS FROM query genes from the database:
    cypher = f"MATCH (k:{target})-[:is_a|part_of|annotates*]->(g:Gene)
    WHERE g.id IN {list(query)} RETURN DISTINCT k.id, k.desc"

    res = neo.run(cypher).to_table()

elif target == 'TU': # EXTRACT TU FROM query genes from the database:
    cypher = f"MATCH (k:{target})-[:harbors]->(g:Gene) WHERE g.id
    IN {list(query)} RETURN DISTINCT k.id, k.desc"
    res = neo.run(cypher).to_table()

elif target == 'Pathway': # EXTRACT PATHWAY FROM query genes from the database:
    cypher = f"MATCH (k:{target})-[:requires]->(g:Gene) WHERE g.id
    IN {list(query)} RETURN DISTINCT k.id, k.desc"
    res = neo.run(cypher).to_table()

elif target == 'InterPro': # EXTRACT Interpro FROM query genes from the database:
    cypher = f"MATCH (k:{target})-[:encoded_in]->(g:Gene) WHERE g.id
    IN {list(query)} RETURN DISTINCT k.id"
    res = neo.run(cypher).to_table()

elif target == 'PubMed': # EXTRACT PubMed FROM query genes from the database:
    cypher = f"MATCH (k:{target})-[:cites]->(g:Gene) WHERE g.id
```

```
    IN {list(query)} RETURN DISTINCT k.id"
    res = neo.run(cypher).to_table()

if param.verbose:
    pprint(f'Cyper request: {cypher}')
    pprint(f'target set: {res}')


#  STATISTICAL TESTS ##########################################################################
results = []
liste_proba = [] # list of dictionaries to store results (probabilities)
dico_proba = {}
desc = []
for r in res:
    # EXTRACT GENES ASSOCIATED (bnumber) WITH CYPHER REQUESTS
    if target == 'Keyword':
        set_id = r['k']['id']
        cypher2 = f"MATCH (t:{target})-->(g:Gene) WHERE t.id = '{set_id}'
        RETURN DISTINCT g.id"
    elif target == 'GOTerm':
        set_id = r[0]
        desc = r[1]
        cypher2 = f"MATCH (t:{target})-[:is_a|part_of|annotates*]->(g:Gene)
        WHERE t.id = '{set_id}' RETURN DISTINCT g.id"
    elif target == 'TU':
        set_id = r[0]
        desc = r[1]
        cypher2 = f"MATCH (t:{target})-[:harbors]->(g:Gene) WHERE t.id = '{set_id}'
        RETURN DISTINCT g.id"
    elif target == 'Pathway':
        set_id = r[0]
        desc = r[1]
        cypher2 = f"MATCH (t:{target})-[:requires]->(g:Gene) WHERE t.id = '{set_id}'
        RETURN DISTINCT g.id"
    elif target == 'InterPro':
        set_id = r[0]
        cypher2 = f"MATCH (t:{target})-[:encoded_in]->(g:Gene) WHERE t.id = '{set_id}'
        RETURN DISTINCT g.id"
    elif target == 'PubMed':
        set_id = r[0]
        cypher2 = f"MATCH (t:{target})-[:cites]->(g:Gene) WHERE t.id = '{set_id}'
        RETURN DISTINCT g.id"

    # EXECUTE CYPHER REQUESTS ACCORDING TO THE TARGET TYPE
    result2 = neo.run(cypher2)

    # EXTRACT UNIQUE GENES ASSOCIATED WITH CYPHER REQUESTS
    elements = set(i[0] for i in result2.to_table())
    common_elements = elements.intersection(query)  # common elements between query and targ
```

```
    k = len(common_elements) # c/k:number of successes in the population = common between qu
    t = len(elements)
    p = len(elements)/n_gene_total # proba of success each essay

    # COVERAGE ##########################################################
    if param.measure == 'coverage':
        proba = 1 - (k/query_size) * (k/t)

    # BINOMIAL TEST #####################################################
    if param.measure == 'binomial':
        # based on success
        # proba = 1 - binom.cdf(len(common_elements)-1, query_size, p) #query_size -
len(common_elements)
        # based on failure : best
        proba = binom.cdf(query_size - k, query_size, 1 - p)

    # HYPERGEOMETRIC TEST ###############################################
    if param.measure == 'hypergeometric':
        proba = 1 - hypergeom.cdf(k - 1, n_gene_total, t, query_size)
        # x = k, M = n_gene_total, t = len(elements), n = query_size
        # pdf = hypergeom.pmf(x, M, n, N)
        # k success parmi n du query/target, N = population size,
        #t = number of target in the population, N = number of query

    # CHI2 ##############################################################
    if param.measure == 'chi2': # adjusted chi square cdf(x, df, loc=0, scale=1)
        # Creation of the contingency table as observed = [[c, q - c], [t - c, g -
q - t + c]]
        observed = [[k, query_size - k], [ t - k, n_gene_total - ( t + query_size -
k)]]
        # chi2_contingency test
        chi2_stat, proba, _, _ = chi2_contingency(observed)


    # if proba <= param.alpha:
    dico_proba = {
                  "id": set_id,
                  "p-value": proba,
                  'common.n':len(common_elements),
                  'target.n': len(elements),
                  'elements.target': elements,
                  'elements.common': common_elements,
                  'desc': desc
                 }
    liste_proba.append(dico_proba)

liste_triee = sorted(liste_proba, key=lambda x: x["p-value"])

if param.adjust:
```

```
    liste_ajust = []

    for i, r in enumerate(liste_triee):
        p_val_ajust = (r["p-value"] * len(liste_triee)) / (i + 1)  # [v2] m = nb de tests, p
        if p_val_ajust <= param.alpha:
            liste_ajust.append(r)

    pprint(f'Number of significative adjusted p-value(0.05): {len(liste_ajust)}')
    for r in liste_ajust:
        print("{}\t{}\t{}/{}\t{}\t{}".format(r['id'], r['p-value'],
        r['common.n'], r['target.n'], r['desc'], ', '.join(r['elements.common'])))

else:
    pprint(f'Number of significative adjusted p-value(0.05): {len(liste_triee)}')
    for r in liste_triee:
        print("{}\t{}\t{}/{}\t{}\t{}".format(r['id'], r['p-value'],
        r['common.n'], r['target.n'], r['desc'], ', '.join(r['elements.common'])))
```

CHAPITRE 9

ANNEXE - ENSEMBLE QUERY 6 - set.6.M2.txt

b0014 b0015 b0058 b0210 b0315 b0385 b0438 b0439 b0457 b0473 b0492 b0631 b0660 b0833 b0834 b0879 b0966 b1025 b1037 b1038 b1039 b1040 b1041 b1042 b1043 b1060 b1114 b1163 b1168 b1274 b1279 b1280 b1285 b1321 b1322 b1341 b1341 b1379 b1380 b1489 b1490 b1522 b1535 b1664 b1707 b1779 b1785 b1786 b1794 b1815 b1829 b1956 b2044 b2045 b2046 b2047 b2048 b2049 b2050 b2051 b2052 b2053 b2054 b2055 b2056 b2057 b2058 b2059 b2060 b2061 b2062 b2067 b2176 b2395 b2503 b2592 b2603 b2604 b2614 b3178 b3179 b3252 b3279 b3400 b3401 b3461 b3497 b3498 b3525 b3529 b3594 b3635 b3686 b3687 b3931 b3932 b3989 b4061 b4140 b4142 b4143 b4174 b4175 b4208 b4312 b4313 b4314 b4315 b4316 b4317 b4318 b4319 b4320 b4484